

Отчёт по лабораторной работе №2

Имитационное моделирование

Кудякова София Андреевна

Содержание

1	Цель работы	6
2	Задание	7
3	Теоретическое введение	8
3.1	Модель SIR	8
3.2	Модель Лотки–Вольтерры	9
4	Выполнение лабораторной работы	10
4.1	Подготовка программного окружения	11
5	Модель SIR	12
5.1	Базовая реализация	12
5.2	Графики модели SIR	13
5.3	Параметрическое исследование модели SIR	16
5.4	Производные файлы модели SIR	17
6	Модель Лотки–Вольтерры	19
6.1	Базовая реализация	19
6.2	Графики модели Лотки–Вольтерры	20
6.3	Параметрическое исследование модели Лотки–Вольтерры	23
6.4	Производные файлы модели Лотки–Вольтерры	24
7	Реализация	25
7.1	Модель SIR	25
8	Модель SIR	26
8.1	Подключение пакетов	26
8.2	Подготовка каталогов	26
8.3	Определение модели SIR	27
8.4	Начальные условия и параметры	27
8.5	Базовое репродуктивное число	28
8.6	Формирование задачи Коши	28
8.7	Численное решение	29
8.8	Подготовка результатов	30
8.9	Вывод параметров модели	31
8.10	Вывод начальных условий	32
8.11	Динамика трёх групп населения	32
8.12	Поиск пика эпидемии	34
8.13	Динамика числа инфицированных	34

8.14	Анализ экспоненциального роста	35
8.15	Доли групп населения	36
8.16	Фазовый портрет	38
8.17	Эффективное репродуктивное число	39
8.18	Динамика эффективного репродуктивного числа	39
8.19	Сохранение графиков	40
8.20	Анализ результатов	42
8.21	Оценка производительности	43
8.22	Параметрическое исследование	44
8.23	Таблица результатов параметрического исследования	46
8.24	Сравнение результатов	46
8.25	Сохранение результата параметрического исследования	48
8.26	Модель Лотки–Вольтерры	49
9	Модель Лотки–Вольтерры	50
9.1	Подключение пакетов	50
9.2	Подготовка каталогов	50
9.3	Определение модели	51
9.4	Параметры модели	52
9.5	Интервал моделирования	52
9.6	Формирование задачи	53
9.7	Численное решение	53
9.8	Подготовка результатов	55
9.9	Производные популяций	57
9.10	Стационарная точка	58
9.11	Вывод параметров	58
9.12	Динамика популяций	60
9.13	Фазовый портрет	61
9.14	Производные популяций	62
9.15	Относительные изменения	63
9.16	Спектральный анализ	65
9.17	Сохранение графиков	67
9.18	Статистический анализ	69
9.19	Параметрическое исследование	70
9.20	Выполнение Jupyter Notebook	73
10	Литературное программирование	75
11	Анализ результатов	76
11.1	Модель SIR	76
11.2	Модель Лотки–Вольтерры	76
12	Выводы	78
	Список литературы	79

Список иллюстраций

4.1	Структура рабочего проекта	10
4.2	Проверка рабочего окружения	11
5.1	Результат выполнения модели SIR	13
5.2	Динамика восприимчивых, инфицированных и выздоровевших	14
5.3	Доля инфицированных во времени	14
5.4	Скорость изменения численности групп	15
5.5	Фазовый портрет модели SIR	15
5.6	Сводный график модели SIR	16
5.7	Параметрическое исследование модели SIR	17
5.8	Генерация производных файлов модели SIR	18
6.1	Результат выполнения модели Лотки–Вольтерры	20
6.2	Динамика численности жертв и хищников	20
6.3	Фазовый портрет модели Лотки–Вольтерры	21
6.4	Производные численности популяций	21
6.5	Относительные изменения численности популяций	22
6.6	Спектральный анализ колебаний численности	22
6.7	Результаты параметрического исследования модели Лотки–Вольтерры	23
6.8	Генерация производных файлов модели Лотки–Вольтерры	24
9.1	Результат выполнения Jupyter Notebook модели SIR	73
9.2	Результат выполнения Jupyter Notebook модели Лотки–Вольтерры	74

Список таблиц

1 Цель работы

Познакомиться с моделями SIR и Лотки–Вольтерры, реализовать их средствами Julia, выполнить численное моделирование и исследовать влияние параметров моделей на получаемые результаты.

2 Задание

В ходе лабораторной работы необходимо:

- создать рабочий каталог для программного кода;
- установить необходимые пакеты;
- выполнить предложенный программный код;
- реализовать модели SIR и Лотки–Вольтерры;
- преобразовать код в литературный стиль;
- сгенерировать из литературного кода:
 - чистый код;
 - Jupyter Notebook;
 - документацию в формате Quarto;
- выполнить код из Jupyter Notebook;
- интегрировать документацию в формате Quarto в отчёт;
- добавить вычисления для набора параметров;
- выполнить параметрическое исследование моделей;
- проанализировать полученные результаты.

3 Теоретическое введение

3.1 Модель SIR

Модель SIR является классической математической моделью распространения инфекционного заболевания в закрытой популяции [kermack1927?].

Название модели образовано от трёх групп населения:

- **Susceptible (S)** — восприимчивые к заболеванию;
- **Infectious (I)** — инфицированные;
- **Recovered (R)** — выздоровевшие.

Общая численность популяции определяется выражением:

$$N = S + I + R.$$

Модель описывается системой дифференциальных уравнений:

$$\frac{dS}{dt} = -\beta c \frac{SI}{N},$$

$$\frac{dI}{dt} = \beta c \frac{SI}{N} - \gamma I,$$

$$\frac{dR}{dt} = \gamma I.$$

Здесь β — вероятность передачи инфекции при контакте, c — среднее число контактов, а γ — скорость выздоровления.

Базовое репродуктивное число определяется выражением:

$$R_0 = \frac{\beta c}{\gamma}.$$

Если $R_0 > 1$, инфекция может распространяться среди популяции.

Модель позволяет исследовать изменение численности восприимчивых, инфицированных и выздоровевших людей во времени, а также определить пик эпидемии и его момент.

3.2 Модель Лотки–Вольтерры

Модель Лотки–Вольтерры описывает взаимодействие двух популяций: жертв и хищников [Lotka1925?,volterra1926?].

Пусть x — численность жертв, а y — численность хищников.

Модель задаётся системой:

$$\frac{dx}{dt} = \alpha x - \beta xy,$$

$$\frac{dy}{dt} = \delta xy - \gamma y.$$

Здесь:

- α — скорость естественного размножения жертв;
- β — коэффициент взаимодействия жертв и хищников;
- δ — коэффициент преобразования потреблённых ресурсов в прирост хищников;
- γ — скорость естественной смертности хищников.

Модель описывает циклическое изменение численности двух взаимодействующих популяций.

При увеличении количества жертв создаются условия для роста популяции хищников. Рост числа хищников, в свою очередь, приводит к уменьшению количества жертв. После уменьшения количества жертв численность хищников также начинает снижаться, что создаёт условия для нового роста популяции жертв.

4 Выполнение лабораторной работы

В ходе выполнения лабораторной работы были реализованы модели SIR и Лотки–Вольтерры с использованием языка программирования Julia.

Для организации проекта использовалась структура Julia-проекта с отдельными каталогами для исходного кода, Jupyter Notebook, документов Quarto и графических результатов.

Основная структура проекта имеет вид:

```
project/
├─ markdown/
├─ notebooks/
├─ plots/
├─ scripts/
├─ src/
├─ Project.toml
└─ Manifest.toml
```

Структура рабочего проекта представлена на рисунке Рисунок 4.1.

```
(base) sofia@MacBook-Pro-Sonya lab02 % ls presentation project report
presentation:
  _quarto.yml
  _resources
  image
  Makefile
  simulation-modeling-lab02-simulation-modeling-lab02-presentation.qmd

project:
  _research
  data
  Manifest.toml
  markdown
  notebooks
  papers
  plots
  Project.toml
  README.md
  scripts
  src
  test

report:
  _extensions
  _quarto.yml
  _resources
  bib
  image
  Makefile
  simulation-modeling-lab02-report.qmd

(base) sofia@MacBook-Pro-Sonya lab02 %
```

Рисунок 4.1: Структура рабочего проекта

4.1 Подготовка программного окружения

Для выполнения лабораторной работы были установлены необходимые пакеты Julia. После настройки проекта была выполнена проверка рабочего окружения.

Результат проверки представлен на рисунке Рисунок 4.2.

```
julia> Pkg.add([
    "Distributions",
    "DifferentialEquations",
    "SimpleDiffEq",
    "Tables",
    "DataFrames",
    "Statistics",
    "LaTeXStrings",
    "Plots",
    "BenchmarkTools",
    "Statistics",
    "FFTW"
])
Resolving package versions...
Installed POMats v0.11.41
Installed HypergeometricFunctions v0.1.20
Installed OffsetArrays v1.17.0
Installed FFTW v1.18.0
Installed StatsFuns v2.1.1
Installed StaticArrays v1.9.20
Installed Roots v3.47.7
Installed NearestNeighbors v0.4.29
Installed Rmath.jl v0.5.2+0
Installed Gamma v1.2.0
Installed Ratios v0.4.5
Installed Distances v0.18.12
Installed AbstractFFTs v1.5.0
Installed StatsPlots v0.15.5
Installed SimpleDiffEq v1.18.0
Installed KernelDensity v0.6.12
Installed Clustering v0.15.0
Installed AbstractTrees v0.4.5
Installed FFM.jl v3.5.12+0
Installed Widgets v0.6.8
Installed MultivariateStats v0.18.5
Installed ChainRulesCore v1.26.1
Installed Interpolations v0.16.3
Installed Rmath v0.9.0
Installed IntegerMathUtils v0.1.4
Installed Aspack.jl v3.5.2+0
Installed Primes v0.5.7
Installed Aspack v0.5.4
Installed FFTA v0.3.1
Installed TableOperations v1.1.0
Installed Observables v0.5.5
Installed FillArrays v1.17.0
Installed WoodburyMatrices v1.1.0
Installed Parameters v0.13.1
Installed AxisAlgorithms v1.1.0
Installed QuadGK v2.11.3
Installed Distributions v0.25.131
Installing artifacts
Updating ~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab02/project/project.toml
[6a4b8f9] + BenchmarkTools v1.6.0
[a73ca708] + DataFrames v1.6.2
[8e6a4032] + DifferentialEquations v8.1.1
[7a1c66a] + FFTW v1.18.0
[96a4f0f] + LaTeXStrings v1.4.1
[91a8bcd] + Plots v1.41.7
[60b0a32c] + SimpleDiffEq v1.18.0
[1874b0d1] + Statistics v1.11.4
```

Рисунок 4.2: Проверка рабочего окружения

5 Модель SIR

5.1 Базовая реализация

Для реализации модели SIR был создан Julia-скрипт `sir_ode.jl`.

В программе задаются параметры модели:

$$\beta = 0.05, \quad c = 10.0, \quad \gamma = 0.25.$$

Начальные условия имеют вид:

$$S_0 = 990, \quad I_0 = 10, \quad R_0 = 0.$$

Для выбранных параметров базовое репродуктивное число равно:

$$R_0 = \frac{0.05 \cdot 10}{0.25} = 2.$$

Средняя продолжительность болезни составляет:

$$\frac{1}{\gamma} = 4$$

дня.

Результат выполнения модели SIR представлен на рисунке Рисунок 5.1.

```
(base) sofia@MacBook-Pro-Sonya project % julia --project=. scripts/sir_ode.jl
Параметры модели SIR:
β = 0.05
с = 10.0
γ = 0.25
R0 = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия:
S0 = 990.0
I0 = 10.0
R0 = 0.0

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===
Пиковое число зараженных = 154.8
Время достижения пика = 19.1 дней
Итоговое число переболевших = 775.7
(base) sofia@MacBook-Pro-Sonya project % █
```

Рисунок 5.1: Результат выполнения модели SIR

В результате базового эксперимента получено максимальное число инфицированных:

$$I_{\max} = 154.8.$$

Пик эпидемии достигается примерно на:

$$t_{\text{peak}} = 19.1$$

дня.

Итоговое число выздоровевших составляет:

$$R(\infty) = 775.7.$$

5.2 Графики модели SIR

В процессе моделирования были построены графики динамики основных переменных модели.

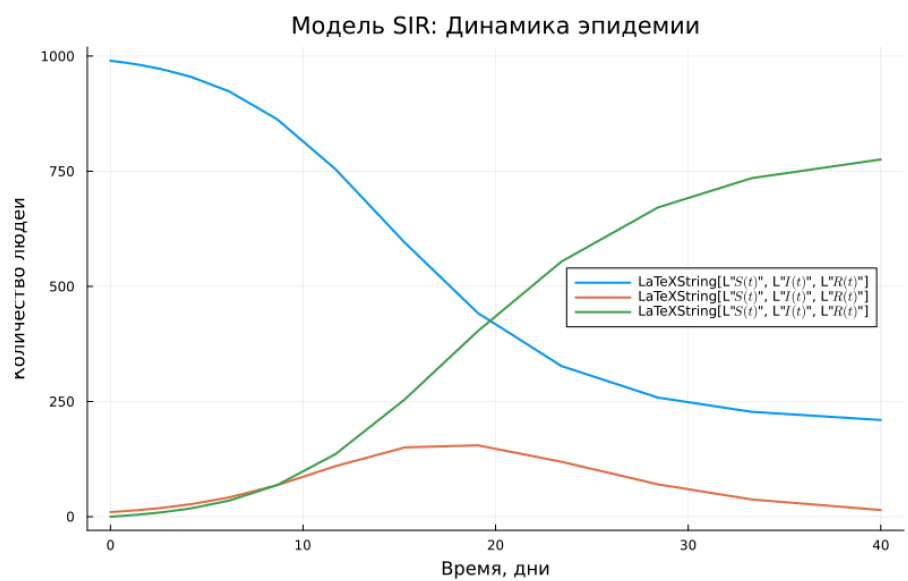


Рисунок 5.2: Динамика восприимчивых, инфицированных и выздоровевших

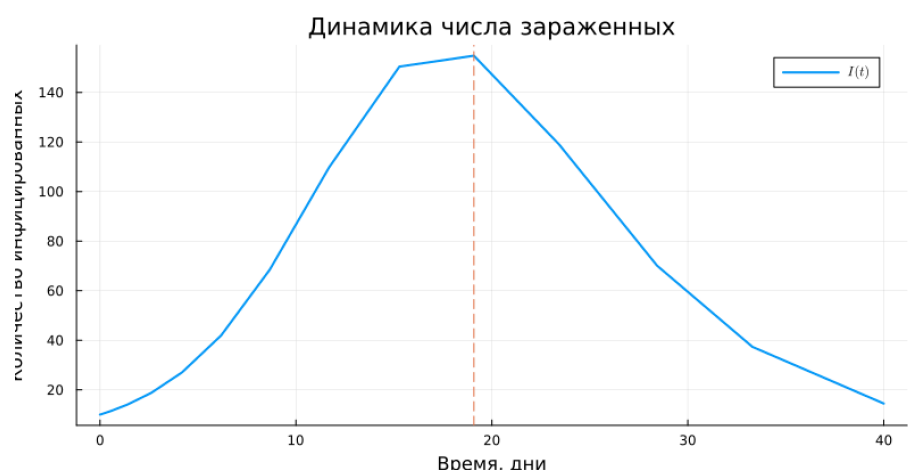


Рисунок 5.3: Доля инфицированных во времени

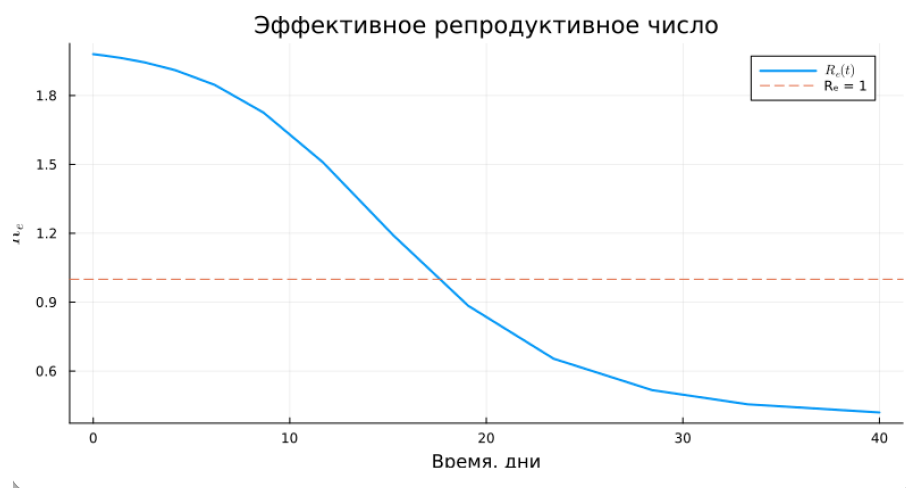


Рисунок 5.4: Скорость изменения численности групп

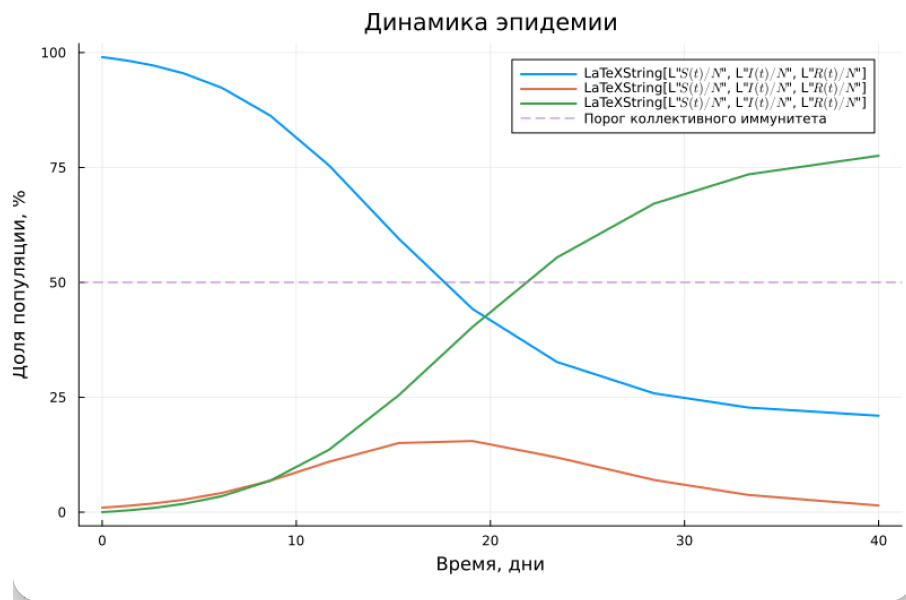


Рисунок 5.5: Фазовый портрет модели SIR

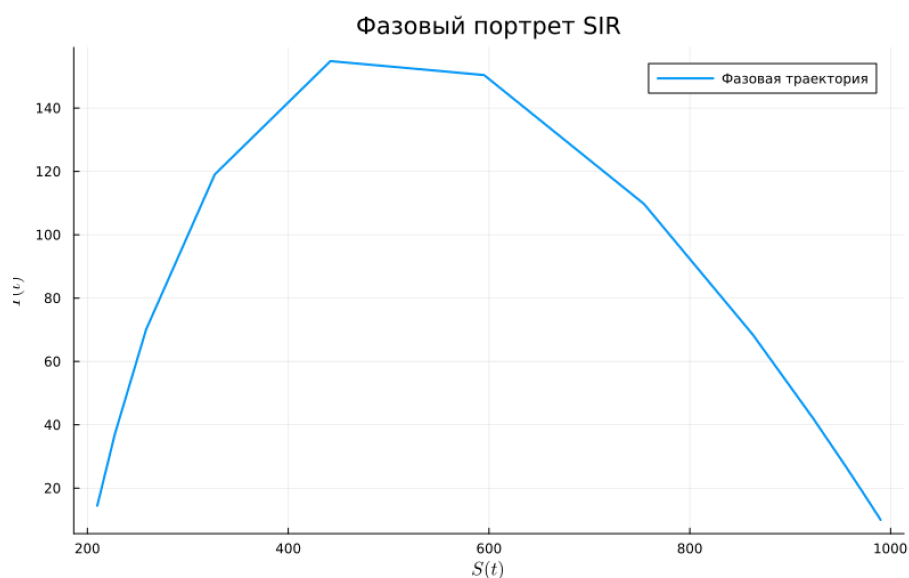


Рисунок 5.6: Сводный график модели SIR

Полученные графики позволяют наблюдать изменение численности трёх групп населения и динамику распространения инфекции.

5.3 Параметрическое исследование модели SIR

Для исследования влияния коэффициента передачи инфекции β было выполнено параметрическое исследование.

Рассматривались следующие значения:

$$\beta \in \{0.03, 0.05, 0.07\}.$$

Для каждого значения параметра рассчитывались:

- максимальное число инфицированных;
- время достижения максимума;
- итоговое число выздоровевших.

Полученные результаты представлены в таблице.

β	$I_{\max}$	t_{peak}	R_{final}
0.03	22.743	36.527	187.764
0.05	154.807	19.083	775.685
0.07	276.454	10.969	923.128

Результат параметрического исследования также представлен на рисунке Рисунок 5.7.

```

julia> cd ~/Documents/Book-120-Sonya-project/01-Open-projects/sir_ode; julia --project=. scripts/sir_ode.jl
((base) sofia@MacBook-Pro-Sonya project % julia --project=. scripts/sir_ode.jl
Параметры модели SIR:
β = 0.05
c = 10.0
γ = 0.25
R0 = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия:
S0 = 990.0
I0 = 10.0
R0 = 0.0

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===
Пиковое число зараженных = 154.8
Время достижения пика = 19.1 дней
Итоговое число переболевших = 775.7

=== ПАРАМЕТРИЧЕСКОЕ ИССЛЕДОВАНИЕ ===
3x4 DataFrame
Row | β      peak_I  peak_time  final_R
    | Float64 Float64  Float64    Float64
1   | 0.03   22.743   36.5266   187.764
2   | 0.05   154.807  19.083    775.685
3   | 0.07   276.454  10.969    923.128

```

Рисунок 5.7: Параметрическое исследование модели SIR

Из полученных результатов видно, что увеличение β приводит к увеличению максимального числа инфицированных и более раннему достижению пика эпидемии.

При $\beta = 0.03$ максимальное число инфицированных составляет около 22.7 человека, а пик достигается примерно на 36.5-м дне.

При $\beta = 0.05$ максимальное число инфицированных составляет около 154.8 человека, а пик достигается примерно на 19.1-м дне.

При $\beta = 0.07$ максимальное число инфицированных составляет около 276.5 человека, а пик достигается примерно на 11-м дне.

Таким образом, увеличение коэффициента β ускоряет распространение инфекции и увеличивает масштаб эпидемии.

5.4 Производные файлы модели SIR

Для литературного кода модели SIR были сформированы производные представления:

- Julia-скрипт;
- Jupyter Notebook;
- документ Quarto.

Файлы модели расположены в следующих каталогах:

```
scripts/sir_ode/sir_ode.jl
notebooks/sir_ode/sir_ode.ipynb
markdown/sir_ode/sir_ode.qmd
```

Результат генерации производных файлов представлен на рисунке Рисунок 5.8.

```
(base) sofia@MacBook-Pro-Sonya project % find scripts/sir_ode notebooks/sir_ode markdown/sir_ode -maxdepth 1 -type f | sort
markdown/sir_ode/sir_ode.qmd
notebooks/sir_ode/sir_ode.ipynb
scripts/sir_ode/sir_ode.jl
```

Рисунок 5.8: Генерация производных файлов модели SIR

6 Модель Лотки–Вольтерры

6.1 Базовая реализация

Для реализации модели Лотки–Вольтерры был создан Julia-скрипт `lv_ode.jl`.

В базовом эксперименте использовались параметры:

$$\alpha = 0.1, \quad \beta = 0.02, \quad \delta = 0.01, \quad \gamma = 0.3.$$

Начальные условия:

$$x_0 = 40, \quad y_0 = 9.$$

Моделирование проводилось на интервале:

$$t \in [0, 200].$$

Стационарная точка определяется выражениями:

$$x^* = \frac{\gamma}{\delta}, \quad y^* = \frac{\alpha}{\beta}.$$

Для выбранных параметров:

$$x^* = 30, \quad y^* = 5.$$

Результат выполнения модели представлен на рисунке Рисунок 6.1.

```

((base) sofia@MacBook-Pro-Sonya project % julia --project=. scripts/lv_ode.jl
Warning: Verbosity toggle: dense_output_saveat
Dense output is incompatible with saveat. Please use the SavingCallback from the Callback Library to mix the two behaviors.
@ OrdinaryDiffEqCore ~/julia/packages/OrdinaryDiffEqCore/JEGPE/src/solve.jl:263
=====
Модель Лотки-Вольтерры
=====

Параметры:
a = 0.1
b = 0.02
c = 0.01
γ = 0.3

Начальные условия:
x0 = 40.0
y0 = 9.0

Стационарная точка:
x* = 30.0
y* = 5.0

Основные статистики:
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2

Моделирование завершено успешно!
(base) sofia@MacBook-Pro-Sonya project % █

```

Рисунок 6.1: Результат выполнения модели Лотки–Вольтерры

6.2 Графики модели Лотки–Вольтерры

Для анализа результатов были построены графики динамики популяций, фазового портрета, производных, относительных изменений и спектрального анализа.

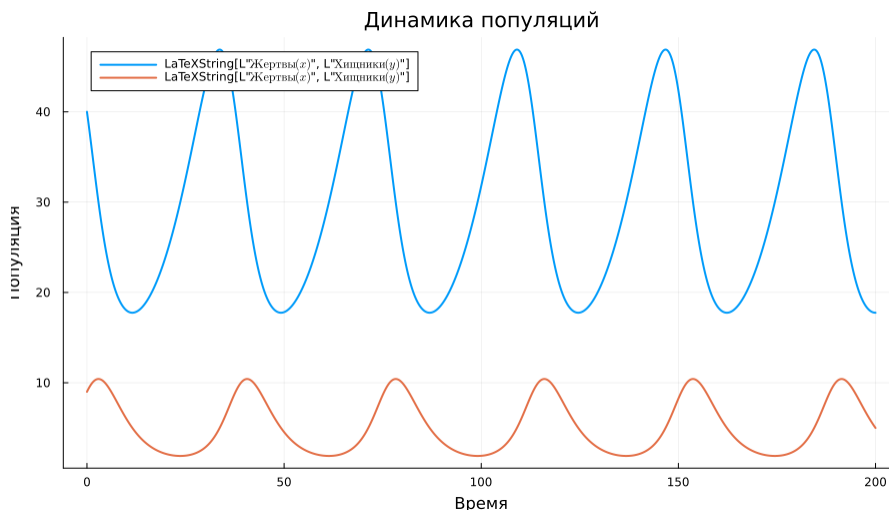


Рисунок 6.2: Динамика численности жертв и хищников

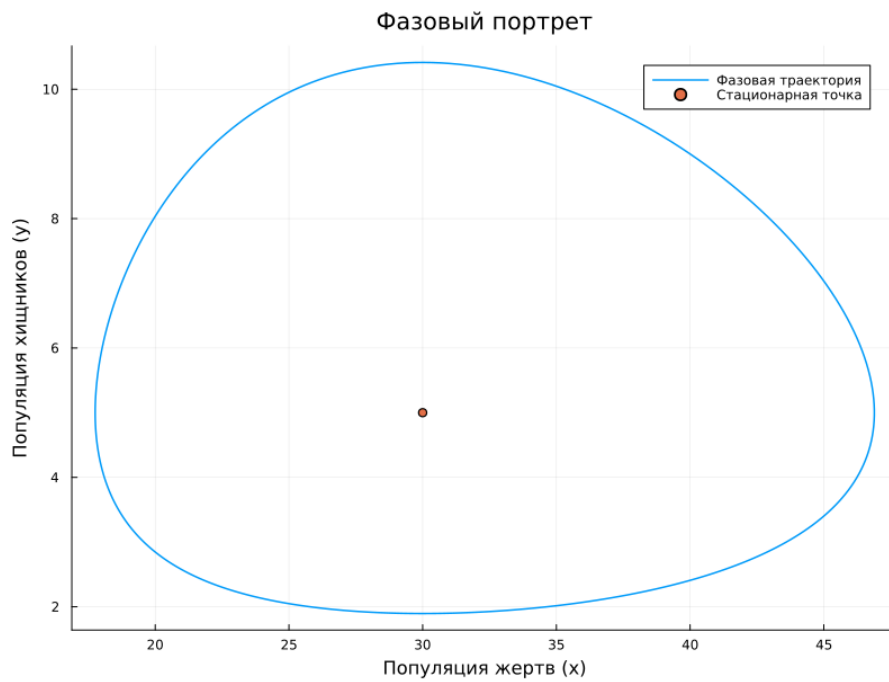


Рисунок 6.3: Фазовый портрет модели Лотки–Вольтерры

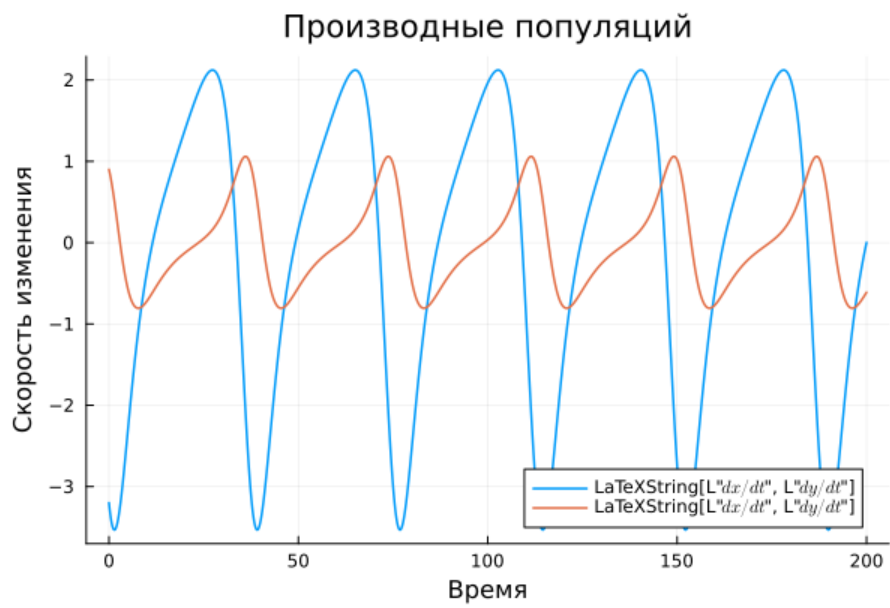


Рисунок 6.4: Производные численности популяций

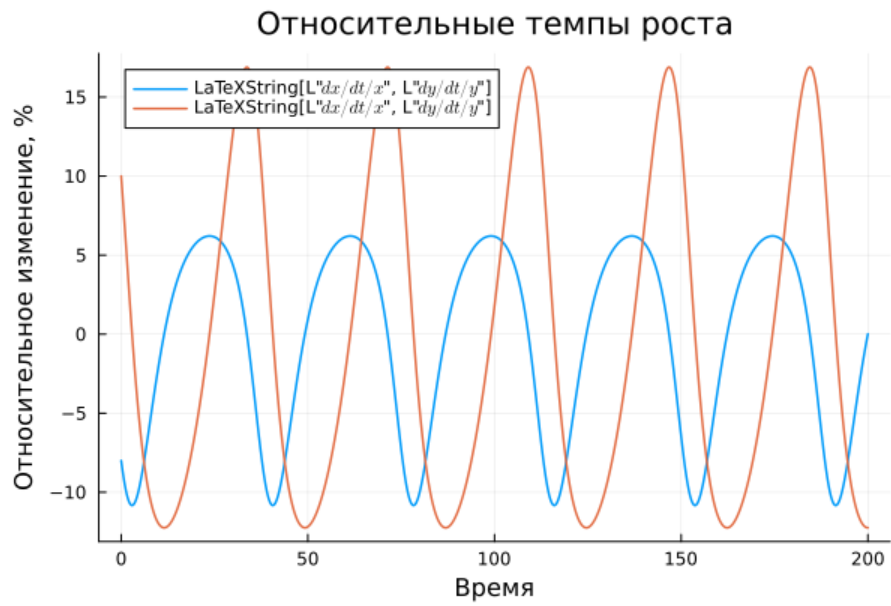


Рисунок 6.5: Относительные изменения численности популяций

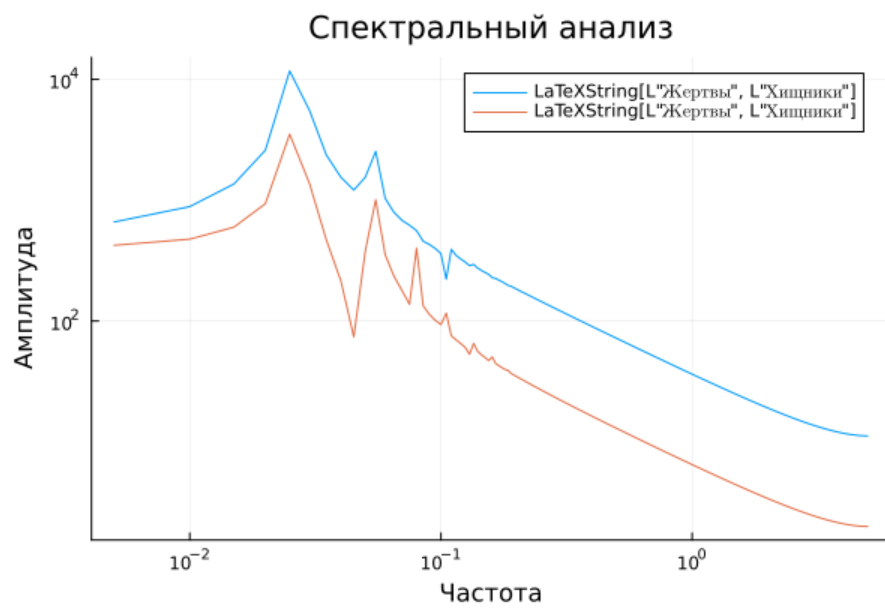


Рисунок 6.6: Спектральный анализ колебаний численности

Полученные результаты демонстрируют характерные циклические изменения численности жертв и хищников.

6.3 Параметрическое исследование модели

Лотки–Вольтерры

Для исследования влияния скорости естественного размножения жертв α было выполнено параметрическое исследование.

Рассматривались следующие значения:

$$\alpha \in \{0.05, 0.1, 0.2\}.$$

Для каждого значения α выполнялось отдельное численное решение системы.

В результате для каждого эксперимента определялись:

- средняя численность жертв;
- средняя численность хищников;
- максимальная численность жертв;
- максимальная численность хищников.

Результаты параметрического исследования выводились в виде таблицы.

```
(base) sofia@MacBook-Pro-Sonya project % julia --project=. scripts/lv_ode.jl
[ Warning: Verbosity toggle: dense_output_saveat
Dense output is incompatible with saveat. Please use the SavingCallback from the Callback Library to mix the two behaviors.
@ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/3EGPE/src/solve.jl:263
=====
Модель Лотки–Вольтерры
=====

Параметры:
α = 0.1
β = 0.02
δ = 0.01
γ = 0.3

Начальные условия:
x0 = 40.0
y0 = 9.0

Стационарная точка:
x* = 30.0
y* = 5.0

Основные статистики:
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2

Моделирование завершено успешно!

Параметрическое исследование Лотки–Вольтерры:
3x5 DataFrame
Row | α      prey_mean predator_mean prey_max predator_max
----|-----
1   | 0.05   28.6783    2.72974   57.4712    9.93873
2   | 0.1     29.6987    5.20289   46.8894   10.4147
3   | 0.2     29.8762   10.1838   40.4219   14.3531
(base) sofia@MacBook-Pro-Sonya project %
```

Рисунок 6.7: Результаты параметрического исследования модели Лотки–Вольтерры

Параметрическое исследование позволяет оценить влияние скорости размножения жертв на динамику обеих популяций.

При изменении α изменяется скорость роста популяции жертв, что также влияет на доступность пищи для хищников и, следовательно, на динамику их численности.

6.4 Производные файлы модели Лотки–Вольтерры

Для литературного кода модели Лотки–Вольтерры были сформированы производные представления:

```
scripts/lv_ode/lv_ode.jl
notebooks/lv_ode/lv_ode.ipynb
markdown/lv_ode/lv_ode.qmd
```

Результат генерации файлов представлен на рисунке Рисунок 6.8.

```
((base) sofia@MacBook-Pro-Sonya project % find scripts/lv_ode notebooks/lv_ode markdown/lv_ode -maxdepth 1 -type f | sort
markdown/lv_ode/lv_ode.qmd
notebooks/lv_ode/lv_ode.ipynb
scripts/lv_ode/lv_ode.jl
```

Рисунок 6.8: Генерация производных файлов модели Лотки–Вольтерры

7 Реализация

7.1 Модель SIR

7.1.1 Базовая реализация

8 Модель SIR

Модель SIR описывает распространение инфекции в популяции, разделённой на три группы: восприимчивые, инфицированные и выздоровевшие.

8.1 Подключение пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using SimpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using BenchmarkTools
```

8.2 Подготовка каталогов

```
script_name = splitext(basename(PROGRAM_FILE))[1]

mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

"/Users/sofia/work/study/2026-1/2026-1==study-simulation-modeling/2026-

8.3 Определение модели SIR

```
function sir_ode!(du, u, p, t)

    S, I, R = u
    β, c, γ = p

    N = S + I + R

    @inbounds begin
        du[1] = -β * c * I / N * S
        du[2] = β * c * I / N * S - γ * I
        du[3] = γ * I
    end

    nothing
end
```

sir_ode! (generic function with 1 method)

8.4 Начальные условия и параметры

```
Δt = 0.1
tmax = 40.0
tspan = (0.0, tmax)

u0 = [990.0, 10.0, 0.0]
```

```
p = [  
    0.05,  
    10.0,  
    0.25  
]
```

```
3-element Vector{Float64}:  
 0.05  
10.0  
 0.25
```

8.5 Базовое репродуктивное число

```
R0 = (p[2] * p[1]) / p[3]
```

```
2.0
```

8.6 Формирование задачи Коши

```
prob_ode = ODEProblem(  
    sir_ode!,  
    u0,  
    tspan,  
    p  
)
```

```
ODEProblem with uType Vector{Float64} and tType Float64. In-place: true  
Non-trivial mass matrix: false  
timespan: (0.0, 40.0)  
u0: 3-element Vector{Float64}:
```

```
990.0
10.0
0.0
```

8.7 Численное решение

```
sol_ode = solve(
    prob_ode,
    dt = ⅈt
)
```

```
retcode: Success
```

```
Interpolation: 3rd order Hermite
```

```
t: 15-element Vector{Float64}:
```

```
0.0
```

```
0.1
```

```
0.5331242537191838
```

```
1.3919008491040001
```

```
2.6102120755226386
```

```
4.1811820758323055
```

```
6.180123810566044
```

```
8.664951930903612
```

```
11.691425331813486
```

```
15.279370667110317
```

```
19.082963490744028
```

```
23.423425193442526
```

```
28.42949373098617
```

```
33.298779173639595
```

```
40.0
```

```
u: 15-element Vector{Vector{Float64}}:
```

```
[990.0, 10.0, 0.0]
```

```
[989.4990153200853, 10.247898072276143, 0.25308660763853125]
[987.1852964117763, 11.391111426852984, 1.4235921613706284]
[981.8312125227972, 14.026022042698546, 4.14276543450428]
[972.1392310589528, 18.757815561654187, 9.102953379393007]
[954.9905500491583, 27.007702518267607, 18.00174743257403]
[923.0706717929863, 41.92976779832721, 34.999560408686385]
[862.6705810342164, 68.49345419063417, 68.83596477514932]
[754.3022993851442, 109.74186185409279, 135.9558387607629]
[595.1014268847736, 150.41363264480682, 254.48494047041947]
[442.0141179101299, 154.80694988491865, 403.1789322049513]
[326.8951167493959, 119.06860394684855, 554.0362793037555]
[258.5085242188966, 70.11052194753069, 671.3809538335727]
[227.58338337085726, 37.328570631579986, 735.0880459975627]
[209.84121402014364, 14.47364496645583, 775.6851410134005]
```

8.8 Подготовка результатов

```
df_ode = DataFrame(
    Tables.table(sol_ode')
)

rename!(
    df_ode,
    ["S", "I", "R"]
)

df_ode[:, :t] = sol_ode.t

df_ode[:, :N] =
    df_ode.S +
```

```
df_ode.I +  
df_ode.R
```

15-element Vector{Float64}:

```
1000.0  
1000.0  
 999.9999999999999  
1000.0  
1000.0  
 999.9999999999999  
1000.0  
 999.9999999999999  
1000.0  
 999.9999999999999  
 999.9999999999998  
1000.0  
1000.0  
1000.0  
1000.0
```

8.9 Вывод параметров модели

```
println("XXXXXXXXXX XXXXXX SIR:")  
println("I = ", p[1])  
println("c = ", p[2])  
println("R = ", p[3])  
  
println(  
    "R0 = ",  
    round(R0, digits = 3)  
)
```

```
println(
    "Средняя продолжительность жизни = ",
    round(1 / p[3], digits = 2),
    " лет"
)
```

Параметры SIR:

$\beta = 0.05$

$c = 10.0$

$\gamma = 0.25$

$R_0 = 2.0$

Средняя продолжительность жизни = 4.0 лет

8.10 Вывод начальных условий

```
println("Параметры: ")
println("S0 = ", u0[1])
println("I0 = ", u0[2])
println("R0 = ", u0[3])
```

Параметры:

$S_0 = 990.0$

$I_0 = 10.0$

$R_0 = 0.0$

8.11 Динамика трёх групп населения

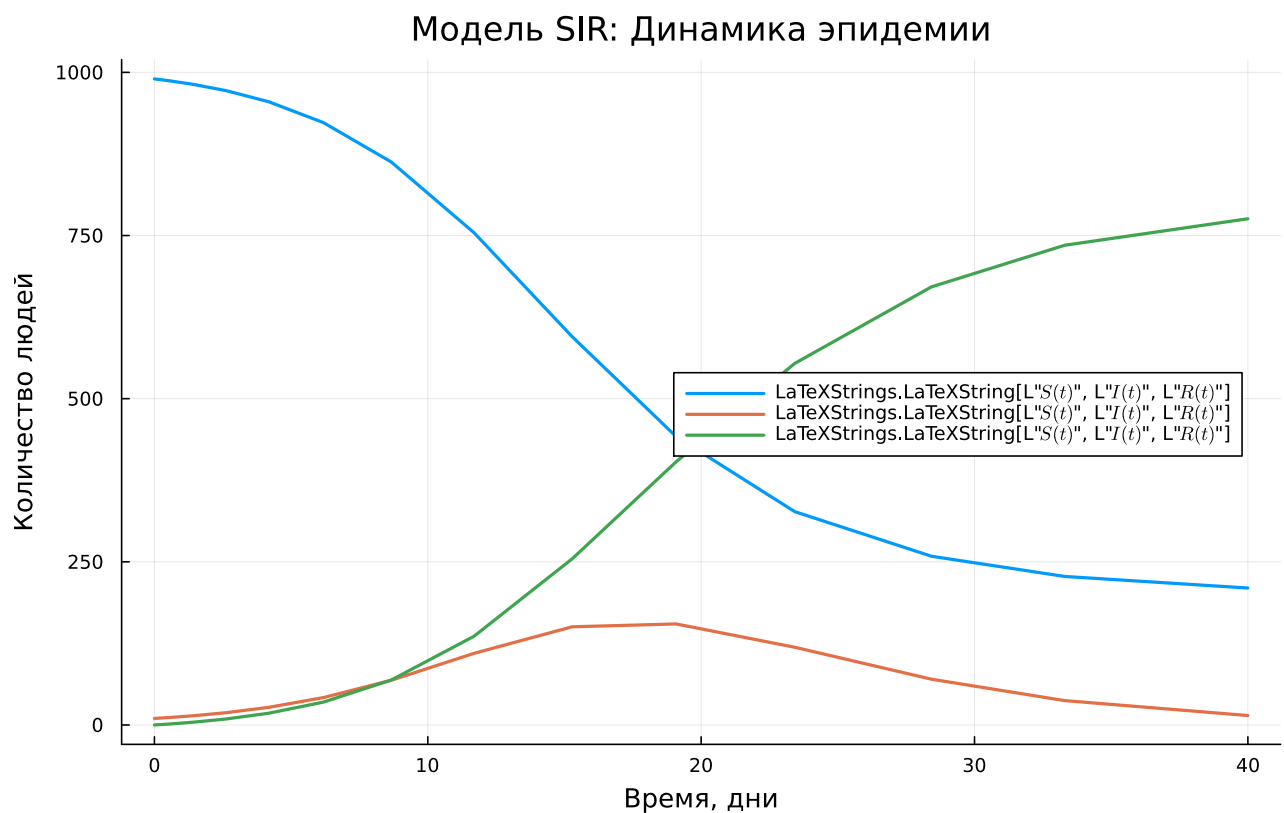
```
plt1 = @df df_ode plot(
    :t,
```



```

[ :S :I :R],
label = [
    L"S(t)"
    L"I(t)"
    L"R(t)"
],
xlabel = "Время, дни",
ylabel = "Количество людей",
title = "Модель SIR: Динамика эпидемии",
linewidth = 2,
legend = :right,
grid = true,
size = (800, 500)
)

```



8.12 Поиск пика эпидемии

```
peak_idx = argmax(df_ode.I)
```

```
peak_time =  
    df_ode.t[peak_idx]
```

```
peak_value =  
    df_ode.I[peak_idx]
```

```
154.80694988491865
```

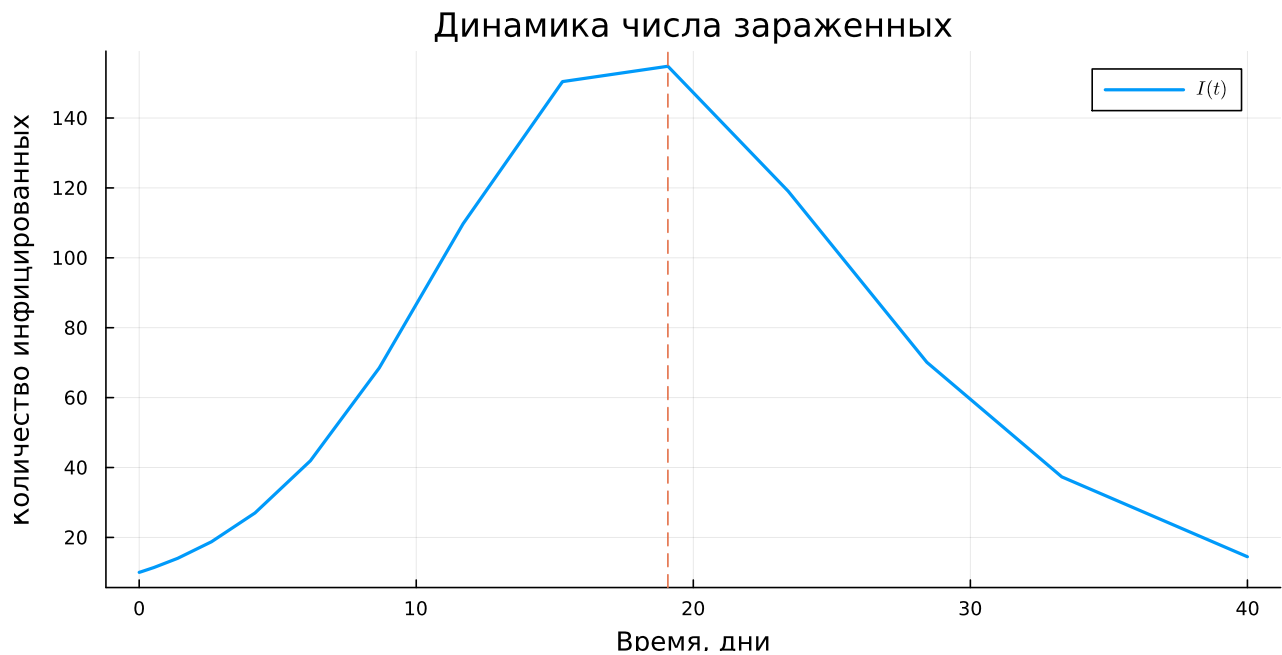
8.13 Динамика числа инфицированных

```
plt2 = @df df_ode plot(  
    :t,  
    :I,  
    label = L"I(t)",  
    xlabel = "time, days",  
    ylabel = "number of infected",  
    title = "Epidemic dynamics",  
    linewidth = 2,  
    grid = true,  
    size = (800, 400)  
)
```

```
vline!(  
    plt2,  
    [peak_time],  
    linestyle = :dash,
```

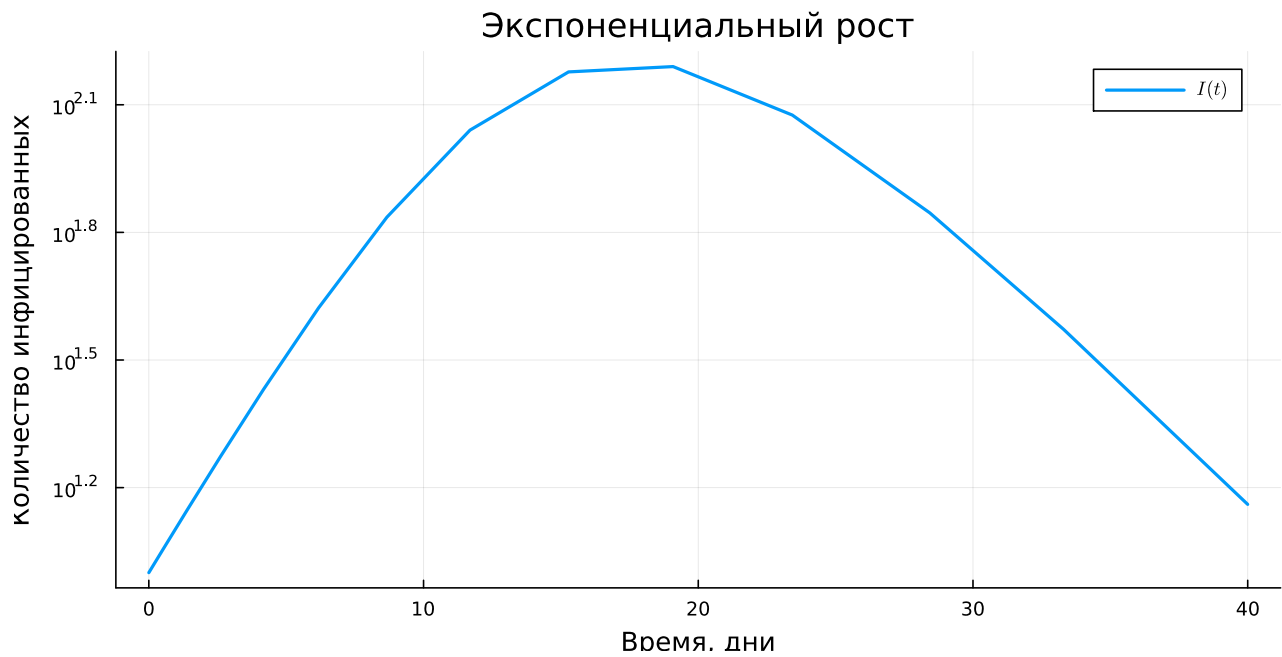
```
label = false
```

```
)
```



8.14 Анализ экспоненциального роста

```
plt3 = @df df_ode plot(
    :t,
    :I,
    label = L"I(t)",
    xlabel = "Время, дни",
    ylabel = "Количество инфицированных",
    title = "Динамика числа зараженных",
    yscale = :log10,
    linewidth = 2,
    grid = true,
    size = (800, 400)
)
```



8.15 Доли групп населения

```
plt4 = @df df_ode plot(
    :t,
    [:S :I :R] ./ df_ode.N .* 100,
    label = [
        L"S(t)/N"
        L"I(t)/N"
        L"R(t)/N"
    ],
    xlabel = "Время, дни",
    ylabel = "Доли групп населения, %",
    title = "Экспоненциальный рост",
    linewidth = 2,
    grid = true,
    size = (800, 500)
)
```

```

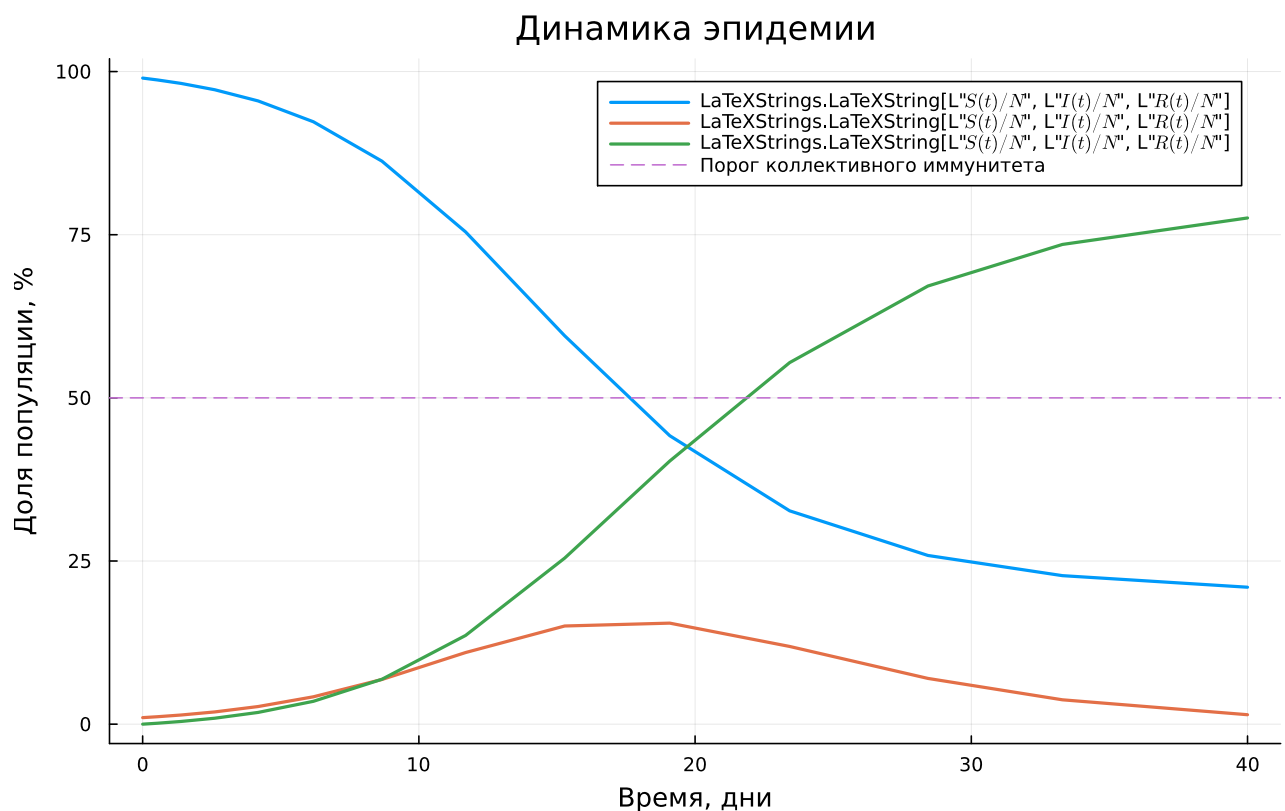
if R0 > 1

    herd_immunity_threshold =
        (1 - 1 / R0) * 100

    hline(
        plt4,
        [herd_immunity_threshold],
        linestyle = :dash,
        label = "XXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXX XXXXXXXXXXXXXXX"
    )

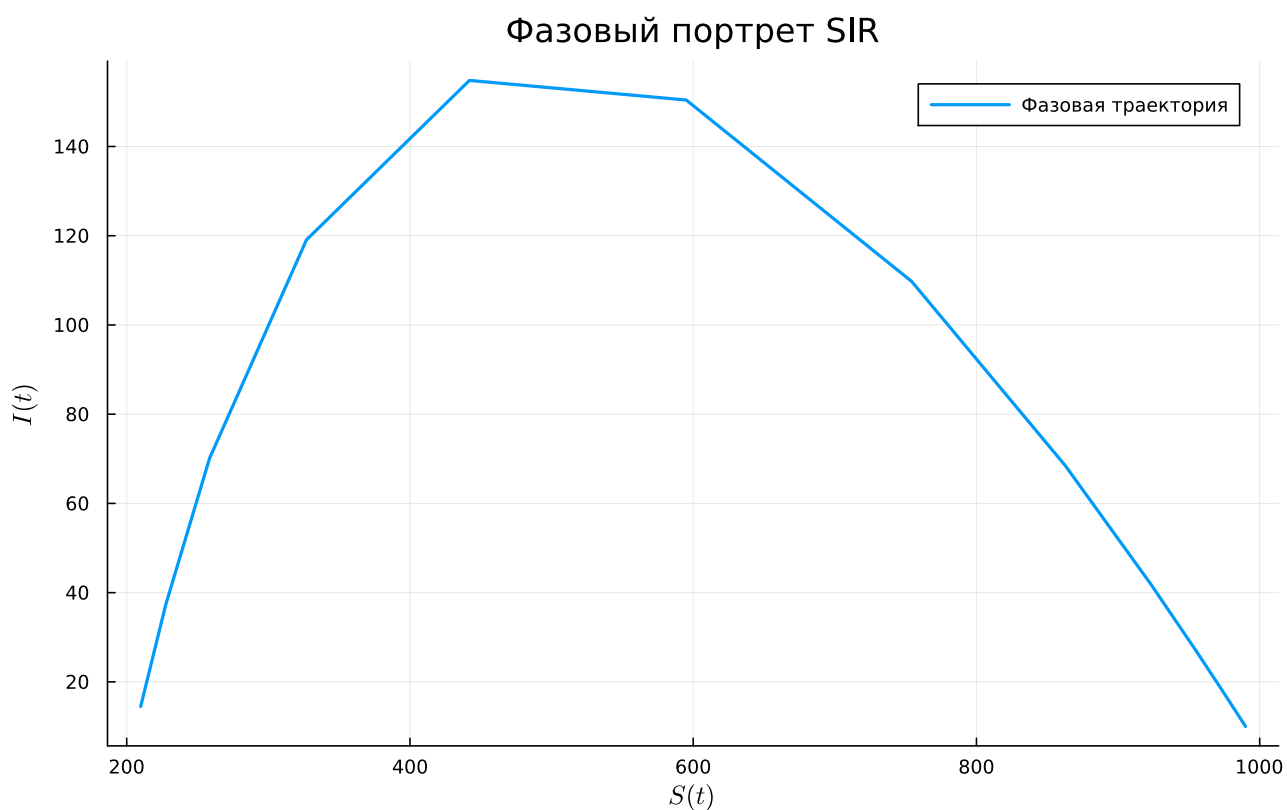
end

```



8.16 Фазовый портрет

```
plt5 = plot(  
    df_ode.S,  
    df_ode.I,  
    label = "XXXXXXXX XXXXXXXXXXXX",  
    xlabel = L"S(t)",  
    ylabel = L"I(t)",  
    title = "XXXXXXXX XXXXXXXXXXXX SIR",  
    linewidth = 2,  
    grid = true,  
    size = (800, 500)  
)
```



8.17 Эффективное репродуктивное число

```
df_ode[!, :Re] =
    R0 .* df_ode.S ./ df_ode.N
```

15-element Vector{Float64}:

1.98
1.9789980306401707
1.9743705928235529
1.9636624250455943
1.9442784621179054
1.909981100098317
1.8461413435859726
1.725341162068433
1.5086045987702885
1.1902028537695473
0.88402823582026
0.6537902334987918
0.5170170484377932
0.4551667667417145
0.41968242804028727

8.18 Динамика эффективного репродуктивного числа

```
plt6 = @df df_ode plot(
    :t,
    :Re,
    label = L"R_e(t)",
    xlabel = "Time (s)",
    ylabel = L"R_e",
    title = "R_e(t) vs Time (s)"
```

```

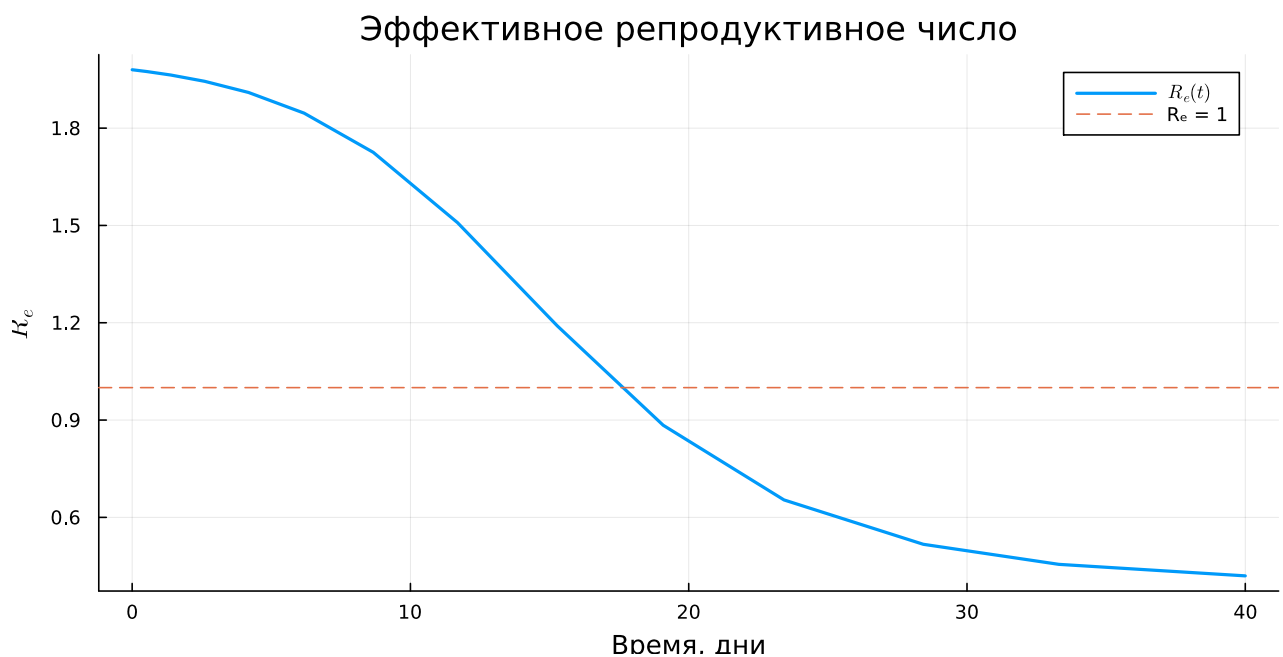
        linewidth = 2,
        grid = true,
        size = (800, 400)
    )

```

```

hline!(
    plt6,
    [1.0],
    linestyle = :dash,
    label = "Re = 1"
)

```



8.19 Сохранение графиков

```

savefig(
    plt1,
    plotsdir(
        script_name,

```



```

        "sir_main.png"
    )
)

savefig(
    plt2,
    plotsdir(
        script_name,
        "sir_infected.png"
    )
)

savefig(
    plt3,
    plotsdir(
        script_name,
        "sir_log_scale.png"
    )
)

savefig(
    plt4,
    plotsdir(
        script_name,
        "sir_percentages.png"
    )
)

savefig(
    plt5,
    plotsdir(

```

```

        script_name,
        "sir_phase_portrait.png"
    )
)

savefig(
    plt6,
    plotsdir(
        script_name,
        "sir_effective_R.png"
    )
)

```

"/Users/sofia/work/study/2026-1/2026-1==study-simulation-modeling/2026-

8.20 Анализ результатов

```

println(
    "\n===  XXXXX  XXXXXXXXXXXX  ==="
)

println(
    "XXXXXX  XXXXX  XXXXXXXXXXXX  = ",
    round(peak_value, digits = 1)
)

println(
    "XXXXXX  XXXXXXXXXXXX  XXXXX  = ",
    round(peak_time, digits = 1),
    "  XXXX"
)

```

```
)

println(
    "XXXXXXXX XXXX XXXXXXXXXXXXXXX = ",
    round(df_ode.R[end], digits = 1)
)
```

```
=== XXXXX XXXXXXXXXXXXXXX ===
XXXXXXXX XXXX XXXXXXXXXXXXXXX = 154.8
XXXXX XXXXXXXXXXXXXXX XXXX = 19.1 XXXX
XXXXXXXX XXXX XXXXXXXXXXXXXXX = 775.7
```

8.21 Оценка производительности

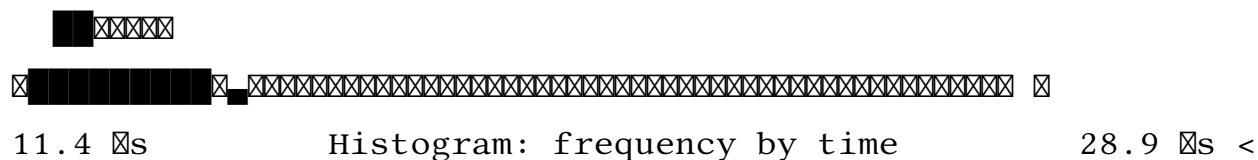
```
@benchmark solve(
    prob_ode,
    dt = 1t
)
```

BenchmarkTools.Trial: 10000 samples with 1 evaluation per sample.

Range (min ... max): 11.435 µs ... 32.652 ms GC (min ... max): 0.00% ... 99.54%

Time (median): 13.104 µs GC (median): 0.00%

Time (mean ± σ): 17.763 µs ± 338.744 µs GC (mean ± σ): 23.37% ± 1.41%



Memory estimate: 42.20 KiB, allocs estimate: 441.

8.22 Параметрическое исследование

Исследуем влияние коэффициента β на развитие эпидемии.

Для сравнения рассматриваем несколько значений β .

```
β_values = [  
    0.03,  
    0.05,  
    0.07  
]  
  
sir_parameter_results = []  
  
for β_value in β_values  
  
    p_test = [  
        β_value,  
        10.0,  
        0.25  
    ]  
  
    prob_test = ODEProblem(  
        sir_ode!,  
        u0,  
        tspan,  
        p_test  
    )  
  
    sol_test = solve(  
        prob_test,  
        dt = βt  
    )
```

```

peak_idx_test =
    argmax(
        [u[2] for u in sol_test.u]
    )

peak_value_test =
    sol_test.u[peak_idx_test][2]

peak_time_test =
    sol_test.t[peak_idx_test]

final_recovered_test =
    sol_test.u[end][3]

push!(
    sir_parameter_results,
    (
        I = I_value,
        peak_I = peak_value_test,
        peak_time = peak_time_test,
        final_R = final_recovered_test
    )
)

```

```
end
```

8.23 Таблица результатов параметрического исследования

```
df_parameter =
    DataFrame(
        sir_parameter_results
    )

println(
    "\n=== XXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXX ==="
)

println(
    df_parameter
)
```

```
=== XXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXXXXXX ===
```

```
3×4 DataFrame
```

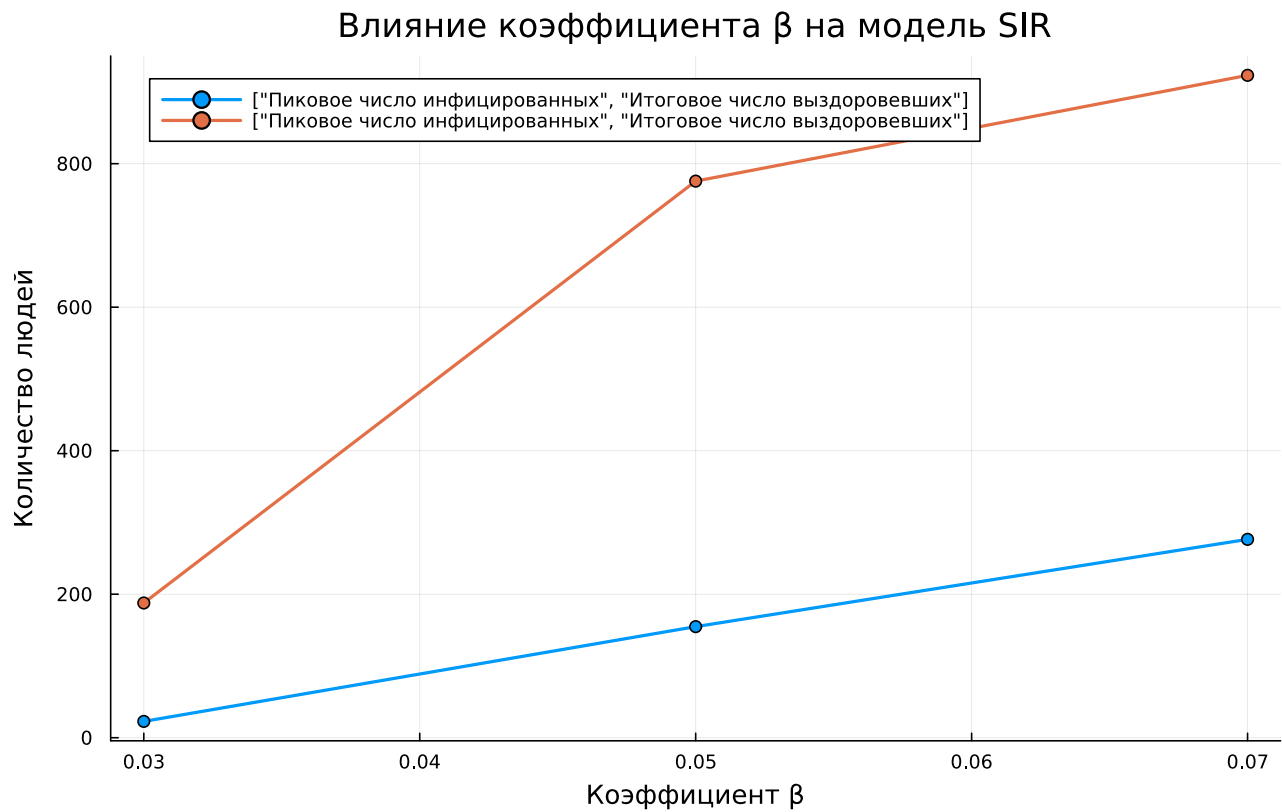
Row	⊠	⊠	peak_I	peak_time	final_R
	⊠	Float64	Float64	Float64	Float64
1	⊠	0.03	22.743	36.5266	187.764
2	⊠	0.05	154.807	19.083	775.685
3	⊠	0.07	276.454	10.969	923.128

8.24 Сравнение результатов

```

plt_parameter = plot(
    df_parameter.%,
    [
        df_parameter.peak_I df_parameter.final_R
    ],
    label = [
        "%s %s %s" % (df_parameter.peak_I, df_parameter.final_R, df_parameter.peak_I)
        "%s %s %s" % (df_parameter.peak_I, df_parameter.final_R, df_parameter.peak_I)
    ],
    marker = :circle,
    xlabel = "%s %s",
    ylabel = "%s %s",
    title = "%s %s %s %s SIR",
    linewidth = 2,
    grid = true,
    size = (800, 500)
)

```



8.25 Сохранение результата параметрического исследования

```
savefig(
    plt_parameter,
    plotsdir(
        script_name,
        "sir_parameter_scan.png"
    )
)

println(
    "\nXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX. "
)
```



```
println(
    "XXXXXXXX XXXXXXXX X: ",
    plotsdir(
        script_name,
        "sir_parameter_scan.png"
    )
)
```

XX.

XXXXXX XXXXXXXX X: /Users/sofia/work/study/2026-1/2026-1==study-simulation-

8.26 Модель Лотки–Вольтерры

8.26.1 Базовая реализация

9 Модель Лотки–Вольтерры

Модель Лотки–Вольтерры описывает взаимодействие двух популяций: жертв и хищников.

x — численность жертв; y — численность хищников.

9.1 Подключение пакетов

Используем DifferentialEquations для решения системы дифференциальных уравнений, DataFrames для обработки результатов и Plots для построения графиков.

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW
```

9.2 Подготовка каталогов

Определяем имя скрипта и создаём каталоги для сохранения результатов и графиков.

```
script_name = splitext(basename(PROGRAM_FILE))[1]
```

```
mkpath(plotsdir(script_name))
```

```
mkpath(datadir(script_name))
```

```
"/Users/sofia/work/study/2026-1/2026-1==study-simulation-modeling/2026-
```

9.3 Определение модели

Система Лотки–Вольтерры имеет вид:

$$dx/dt = \alpha x - \beta xy$$

$$dy/dt = \delta xy - \gamma y$$

где:

α — скорость естественного роста жертв; β — интенсивность взаимодействия жертв и хищников; δ — эффективность превращения добычи в прирост хищников; γ — скорость естественного уменьшения численности хищников.

```
function lotka_volterra!(du, u, p, t)
```

```
    x, y = u
```

```
    α, β, δ, γ = p
```

```
    @inbounds begin
```

```
        du[1] =
```

```
            α * x -
```

```
            β * x * y
```

```
        du[2] =
```

```
            δ * x * y -
```

```

        x * y

    end

    nothing
end

```

lotka_volterra! (generic function with 1 method)

9.4 Параметры модели

Задаём значения параметров модели и начальные численности популяций.

```

p_lv = [
    0.1,
    0.02,
    0.01,
    0.3
]

u0_lv = [
    40.0,
    9.0
]

```

```

2-element Vector{Float64}:
 40.0
  9.0

```

9.5 Интервал моделирования

Исследуем динамику популяций на интервале от 0 до 200 единиц времени.

```
tspan_lv = (
    0.0,
    200.0
)

dt_lv = 0.01
```

```
0.01
```

9.6 Формирование задачи

Создаём задачу Коши для системы дифференциальных уравнений.

```
prob_lv = ODEProblem(
    lotka_volterra!,
    u0_lv,
    tspan_lv,
    p_lv
)
```

```
ODEProblem with uType Vector{Float64} and tType Float64. In-place: true
Non-trivial mass matrix: false
timespan: (0.0, 200.0)
u0: 2-element Vector{Float64}:
 40.0
  9.0
```

9.7 Численное решение

Решаем систему численно с помощью метода Tsit5.

Сохраняем результаты через каждые 0.1 единицы времени.

```
sol_lv = solve(
    prob_lv,
    Tsit5(),
    dt=dt_lv,
    reltol=1e-8,
    abstol=1e-10,
    saveat=0.1,
    dense=true
)
```

```
└ Warning: Verbosity toggle: dense_output_saveat
```

```
⊠ Dense output is incompatible with saveat. Please use the SavingCallback
```

```
└ @ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/JEGPE/src/solve.jl:100
```

```
retcode: Success
```

```
Interpolation: specialized 4th order "free" interpolation
```

```
t: 2001-element Vector{Float64}:
```

```
0.0
```

```
0.1
```

```
0.2
```

```
0.3
```

```
0.4
```

```
0.5
```

```
0.6
```

```
0.7
```

```
0.8
```

```
0.9
```

```
⊠
```

```
199.2
```

```
199.3
```

```
199.4
```

```
199.5
```

199.6

199.7

199.8

199.9

200.0

u: 2001-element Vector{Vector{Float64}}:

[40.0, 9.0]

[39.67773201890384, 9.088990237207637]

[39.35113789503168, 9.175882726306998]

[39.02053587808482, 9.260562174341327]

[38.68624820281499, 9.342916311942014]

[38.34860018759226, 9.422836265415745]

[38.007919320045765, 9.5002169187062]

[37.664534369328166, 9.574957250330058]

[37.31877446654823, 9.646960666854817]

[36.9709682176784, 9.716135308317774]

⊠

[17.824353144929315, 5.508019318506856]

[17.807442037033756, 5.441315154551648]

[17.792907686320664, 5.375334300040311]

[17.780718473179327, 5.310082577777411]

[17.770843588540227, 5.24556508481173]

[17.76325301314863, 5.181786245480946]

[17.75791751745926, 5.118749811721092]

[17.754808657966564, 5.056458873592464]

[17.75389874795971, 4.994915937053478]

9.8 Подготовка результатов

Переносим результаты решения в DataFrame, чтобы упростить дальнейший анализ.

```
df_lv = DataFrame()

df_lv[!, :t] =
    sol_lv.t

df_lv[!, :prey] =
    [u[1] for u in sol_lv.u]

df_lv[!, :predator] =
    [u[2] for u in sol_lv.u]
```

2001-element Vector{Float64}:

```
9.0
9.088990237207637
9.175882726306998
9.260562174341327
9.342916311942014
9.422836265415745
9.5002169187062
9.574957250330058
9.646960666854817
9.716135308317774
⊠
5.508019318506856
5.441315154551648
5.375334300040311
5.310082577777411
5.24556508481173
5.181786245480946
5.118749811721092
5.056458873592464
```


4.994915937053478

9.9 Производные популяций

Рассчитываем скорость изменения численности жертв и хищников.

```
df_lv[!, :dprey_dt] =  
    p_lv[1] .* df_lv.prey .-  
    p_lv[2] .* df_lv.prey .* df_lv.predator  
  
df_lv[!, :dpredator_dt] =  
    p_lv[3] .* df_lv.prey .* df_lv.predator .-  
    p_lv[4] .* df_lv.predator
```

2001-element Vector{Float64}:

```
0.90000000000000004  
0.8796081183812872  
0.8580494468233599  
0.8353523334488111  
0.8115489002365717  
0.7866749261310111  
0.7607697060793308  
0.7338758893000268  
0.706039294083721  
0.677308704329461  
⊠  
-0.6706369809304658  
-0.6634355041663761  
-0.6561720201747897  
-0.648853939485278  
-0.6414883588865488  
-0.6340820722589682
```

```
-0.6266415740257949  
-0.6191730642026259  
-0.611682463105866
```

9.10 Стационарная точка

Для модели Лотки–Вольтерры внутренняя стационарная точка определяется выражениями:

$$x^* = \gamma / \delta$$

$$y^* = \alpha / \beta$$

```
x_star =  
    p_lv[4] / p_lv[3]  
  
y_star =  
    p_lv[1] / p_lv[2]
```

```
5.0
```

9.11 Вывод параметров

Выводим основные параметры и начальные условия используемой модели.

```
println("="^60)  
  
println(  
    "XXXXXXXX XXXXX-XXXXXXXX"  
)  
  
println("="^60)  
  
println("\nXXXXXXXXXX: ")
```

```
println("x = ", p_lv[1])

println("x = ", p_lv[2])

println("x = ", p_lv[3])

println("x = ", p_lv[4])

println("\nXXXXXXXXXX XXXXX:")

println("x0 = ", u0_lv[1])

println("y0 = ", u0_lv[2])

println("\nXXXXXXXXXXXX XXXX:")

println("x* = ", x_star)

println("y* = ", y_star)
```

```
=====
XXXXXX XXXX-XXXXXXXXXX
=====
```

XXXXXXXXXX:

x = 0.1

x = 0.02

x = 0.01

x = 0.3

XXXXXXXXXX XXXXX:

```
x0 = 40.0
```

```
y0 = 9.0
```

```
XXXXXXXXXXXXXXXXXXXX XXXXXX:
```

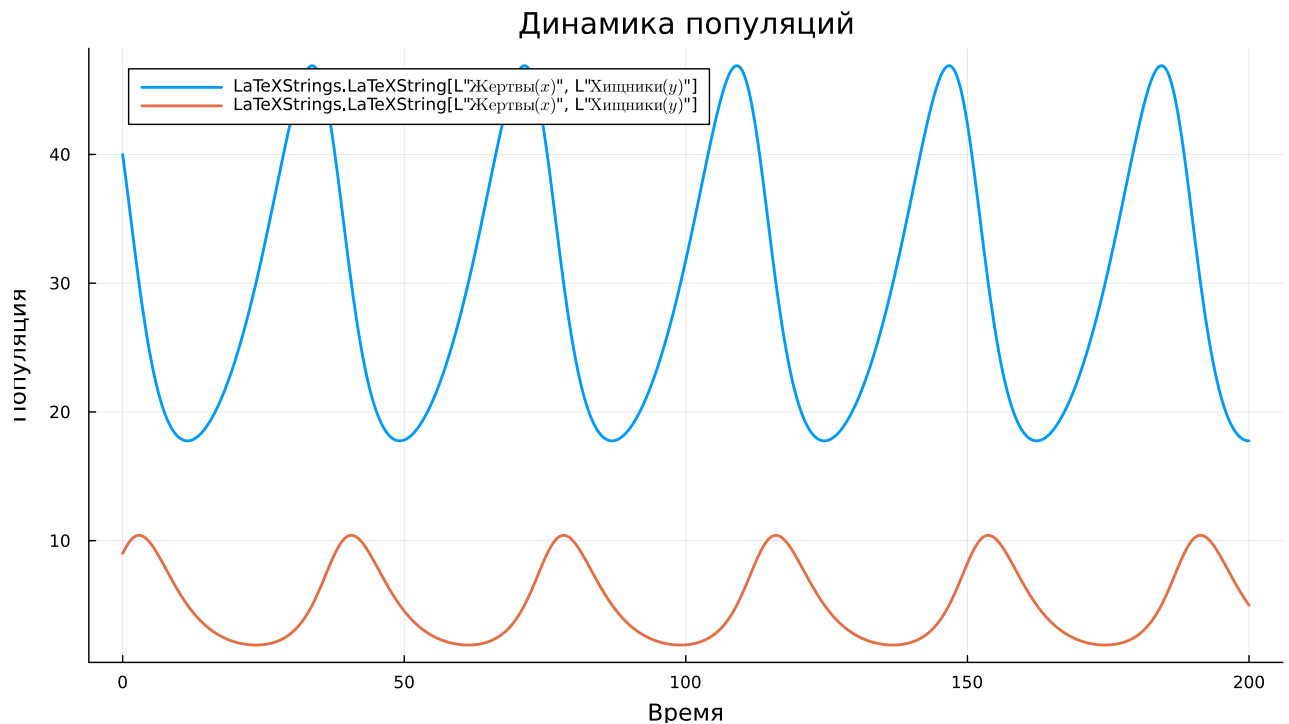
```
x* = 30.0
```

```
y* = 5.0
```

9.12 Динамика популяций

Строим график изменения численности жертв и хищников во времени.

```
plt1 = plot(  
    df_lv.t,  
    [df_lv.prey df_lv.predator],  
    label=[  
        L"XXXXXXXX (x)"  
        L"XXXXXXXX (y)"  
    ],  
    xlabel="XXXXXX",  
    ylabel="XXXXXXXXXXXX",  
    title="XXXXXXXXXXXX XXXXXXXXXXXXXXX",  
    linewidth=2,  
    grid=true,  
    size=(900, 500)  
)
```



9.13 Фазовый портрет

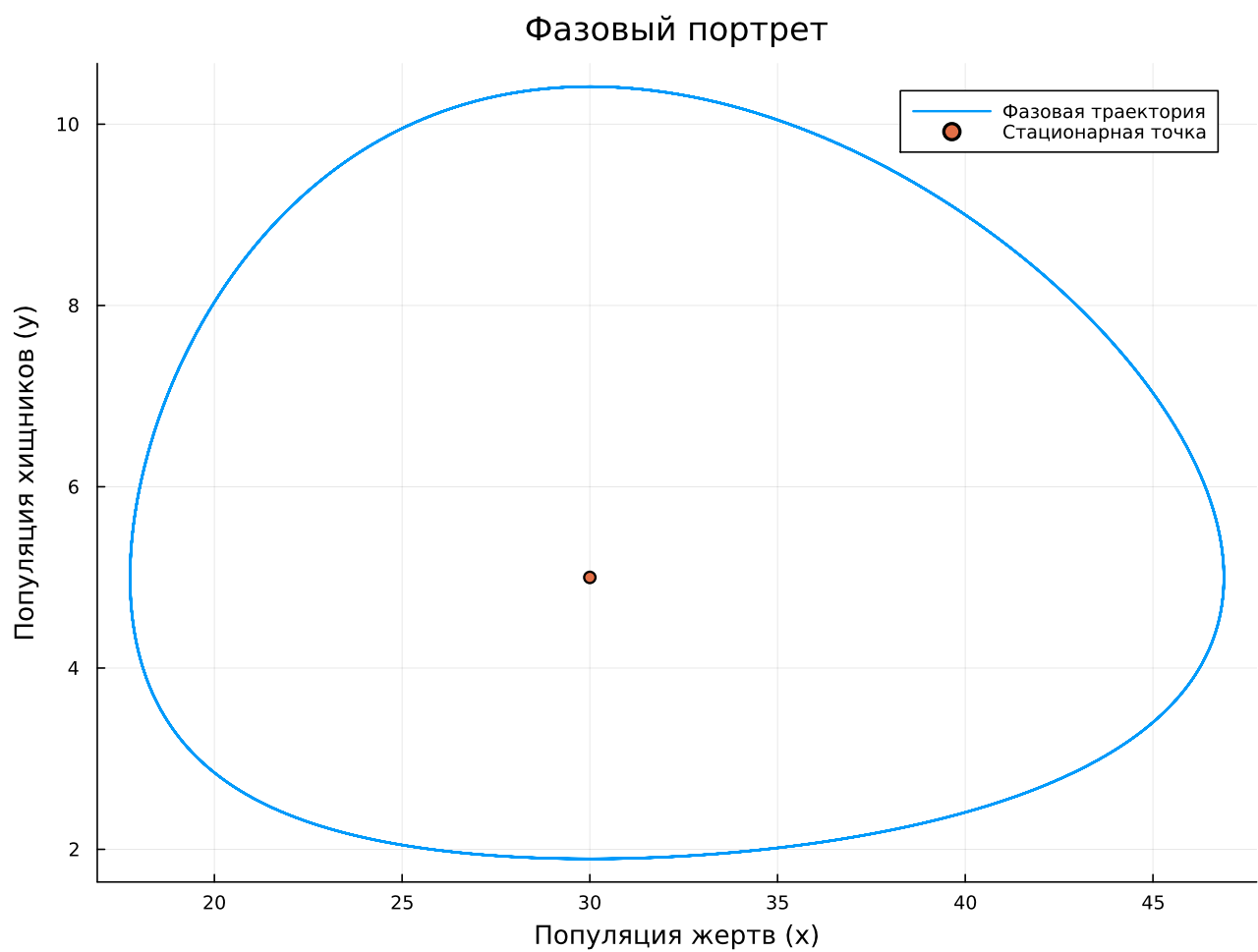
Строим зависимость численности хищников от численности жертв.

Дополнительно отмечаем стационарную точку.

```
plt2 = plot(
    df_lv.prey,
    df_lv.predator,
    label="Жертвы (x), Хищники (y)",
    xlabel="Жертвы (x)",
    ylabel="Хищники (y)",
    title="Динамика популяций",
    linewidth=1.5,
    grid=True,
    size=(800, 600)
)

scatter!
```

```
plt2,
[x_star],
[y_star],
label="XXXXXXXXXXXX XXXX"
)
```



9.14 Производные популяций

Отображаем скорость изменения численности обеих популяций.

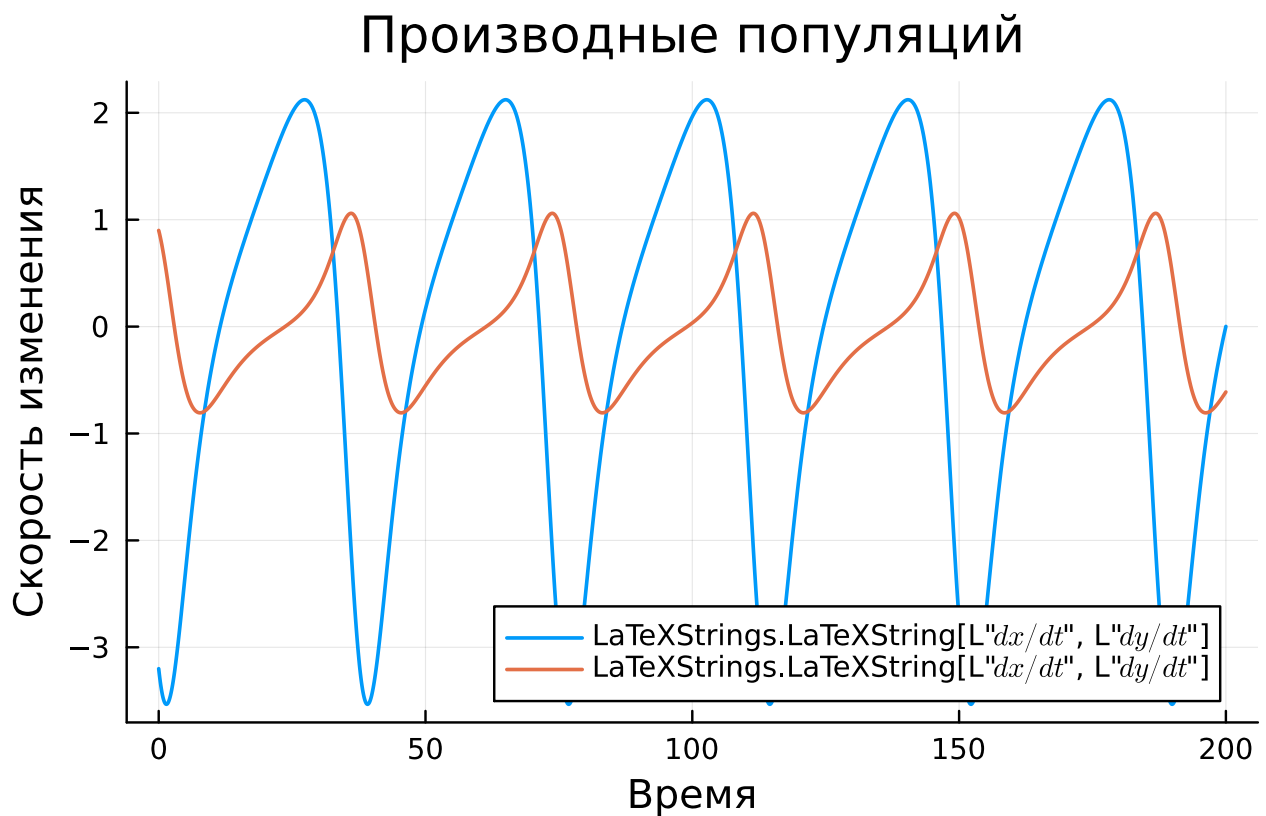
```
plt3 = plot(
    df_lv.t,
    [df_lv.dprey_dt df_lv.dpredator_dt],
    label=[
```

```

        L"dx/dt"
        L"dy/dt"

    ],
    xlabel="Время",
    ylabel="Скорость изменения",
    title="Производные популяций",
    linewidth=1.5,
    grid=True
)

```



9.15 Относительные изменения

Рассчитываем относительные темпы изменения численности жертв и хищников.

```

df_lv[!, :prey_pct_change] =
    df_lv.dprey_dt ./ df_lv.prey .* 100

```

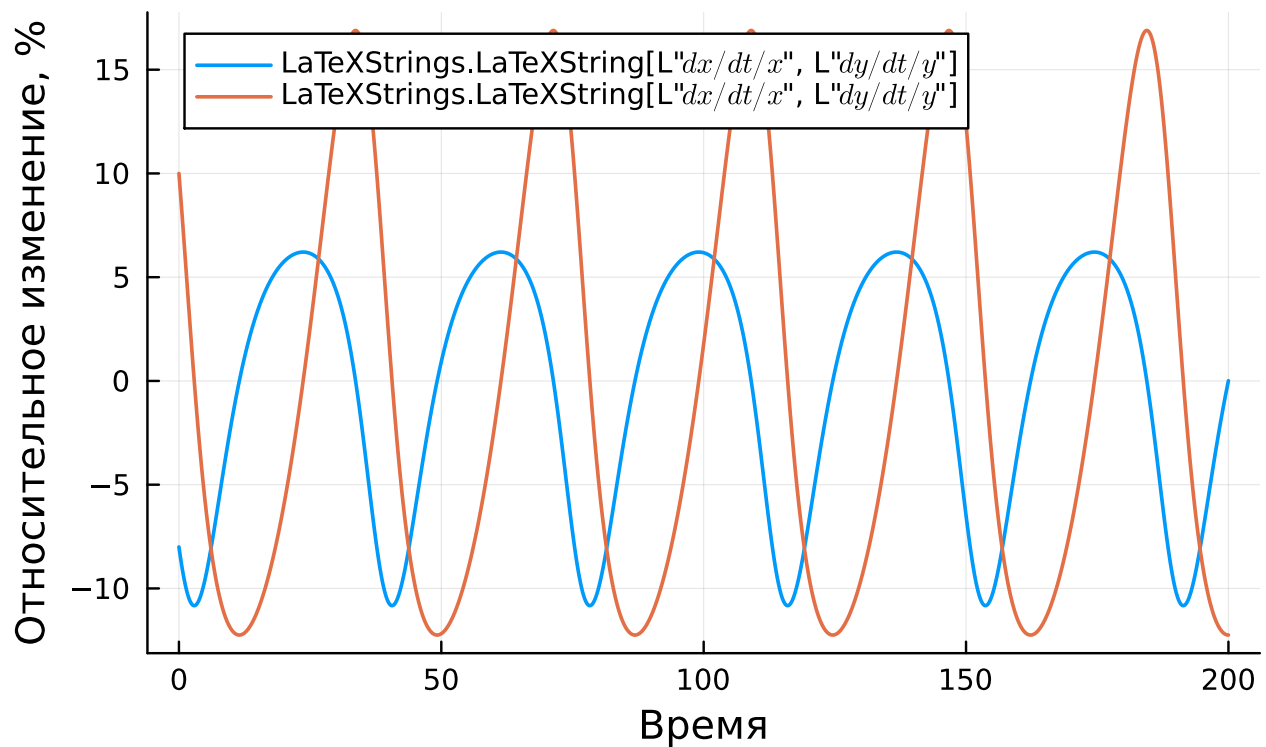
```

df_lv[!, :predator_pct_change] =
    df_lv.dpredator_dt ./ df_lv.predator .* 100

plt4 = plot(
    df_lv.t,
    [
        df_lv.prey_pct_change df_lv.predator_pct_change
    ],
    label=[
        L"dx/dt / x"
        L"dy/dt / y"
    ],
    xlabel="Time",
    ylabel="Relative Change, %",
    title="Prey and Predator Relative Change",
    linewidth=1.5,
    grid=true
)

```


Относительные темпы роста



9.16 Спектральный анализ

Используем быстрое преобразование Фурье для исследования периодичности колебаний численности популяций.

```
function compute_fft(signal, dt)
```

```
    n = length(signal)
```

```
    spectrum =  
        abs.(rfft(signal))
```

```
    freq =  
        rfftfreq(n, 1 / dt)
```

```
    return freq, spectrum
```

```
end
```

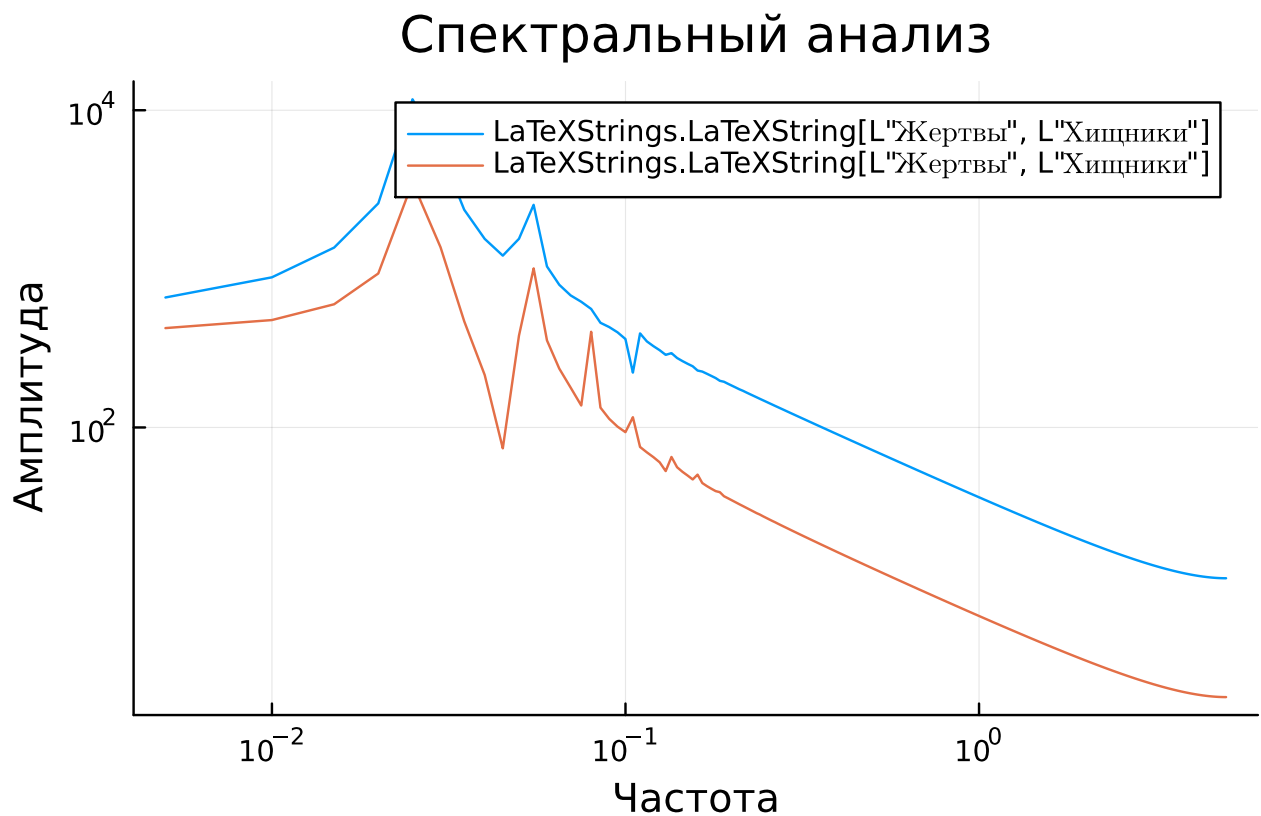
```
freq_pre, spectrum_pre =
```

```
    compute_fft(  
        df_lv.prey .- mean(df_lv.prey),  
        0.1  
    )
```

```
freq_predator, spectrum_predator =
```

```
    compute_fft(  
        df_lv.predator .- mean(df_lv.predator),  
        0.1  
    )
```

```
plt5 = plot(  
    freq_pre[2:end],  
    [  
        spectrum_pre[2:end] spectrum_predator[2:end]  
    ],  
    label=[  
        L"XXXXXXXX"  
        L"XXXXXXXX"  
    ],  
    xlabel="XXXXXXXX",  
    ylabel="XXXXXXXXXX",  
    title="XXXXXXXXXXXX XXXXXX",  
    xscale=:log10,  
    yscale=:log10,  
    grid=true  
)
```



9.17 Сохранение графиков

Сохраняем результаты визуализации в каталог plots проекта.

```
savefig(
    plt1,
    plotsdir(
        script_name,
        "lv_dynamics.png"
    )
)

savefig(
    plt2,
    plotsdir(
        script_name,
```

```

        "lv_phase_portrait.png"
    )
)

savefig(
    plt3,
    plotsdir(
        script_name,
        "lv_derivatives.png"
    )
)

savefig(
    plt4,
    plotsdir(
        script_name,
        "lv_relative_changes.png"
    )
)

savefig(
    plt5,
    plotsdir(
        script_name,
        "lv_spectrum.png"
    )
)

```

"/Users/sofia/work/study/2026-1/2026-1==study-simulation-modeling/2026-

9.18 Статистический анализ

Рассчитываем минимальные, максимальные и средние значения численности обеих популяций.

```
println("\nXXXXXXXXXX XXXXXXXXXXXX:")

println(
    "XXXXXX: min = ",
    round(
        minimum(df_lv.prey),
        digits=2
    ),
    ", max = ",
    round(
        maximum(df_lv.prey),
        digits=2
    ),
    ", mean = ",
    round(
        mean(df_lv.prey),
        digits=2
    )
)

println(
    "XXXXXX: min = ",
    round(
        minimum(df_lv.predator),
        digits=2
    ),
    ", max = ",
```

```

round(
    maximum(df_lv.predator),
    digits=2
),
", mean = ",
round(
    mean(df_lv.predator),
    digits=2
)
)

println(
    "\nXXXXXXXXXXXXXXXX XXXXXXXXXXX XXXXXXXX!"
)

```

XXXXXXXXXX XXXXXXXXXXXXX:

XXXXXX: min = 17.75, max = 46.89, mean = 29.71

XXXXXXXX: min = 1.9, max = 10.41, mean = 5.2

XXXXXXXXXXXXXXXXXXXXXXXX XXXXXXXXXXX XXXXXXXX!

9.19 Параметрическое исследование

Исследуем влияние параметра α , отвечающего за естественный рост популяции жертв.

Рассматриваем несколько значений α .

```

α_values = [
    0.05,
    0.1,
    0.2

```

```

]

lv_parameter_results = []

for x_value in x_values
    p_test = [
        x_value,
        0.02,
        0.01,
        0.3
    ]

    prob_test = ODEProblem(
        lotka_volterra!,
        u0_lv,
        tspan_lv,
        p_test
    )

    sol_test = solve(
        prob_test,
        Tsit5(),
        dt=dt_lv,
        saveat=0.1
    )

    prey_values = [u[1] for u in sol_test.u]
    predator_values = [u[2] for u in sol_test.u]

    push!(
        lv_parameter_results,

```


9.20 Выполнение Jupyter Notebook

После генерации производных файлов были выполнены Jupyter Notebook для обеих моделей.

Для модели SIR использовался файл:

```
notebooks/sir_ode/sir_ode.ipynb
```

Для модели Лотки–Вольтерры:

```
notebooks/lv_ode/lv_ode.ipynb
```

Результат выполнения Jupyter Notebook модели SIR представлен на рисунке Рисунок 9.1.

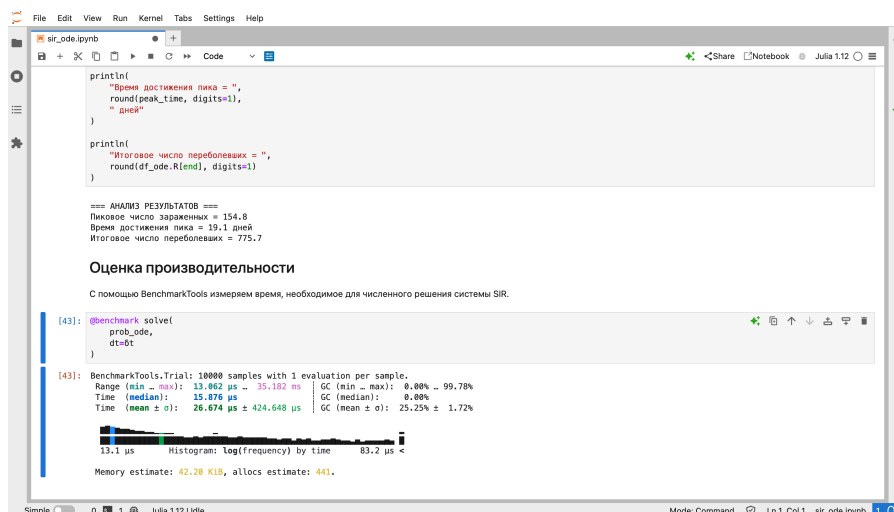


Рисунок 9.1: Результат выполнения Jupyter Notebook модели SIR

Результат выполнения Jupyter Notebook модели Лотки–Вольтерры представлен на рисунке Рисунок 9.2.

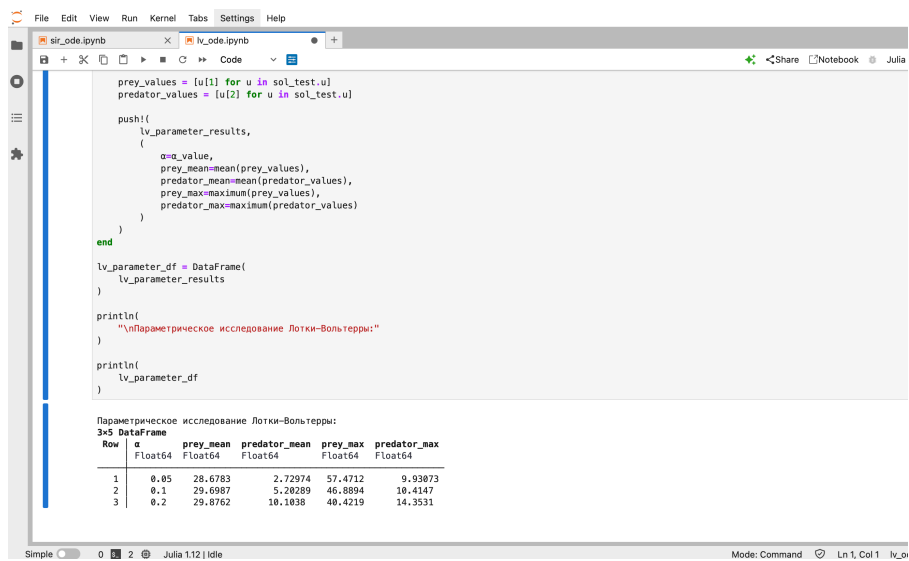


Рисунок 9.2: Результат выполнения Jupyter Notebook модели Лотки–Вольтерры

10 Литературное программирование

Для преобразования программного кода использовался литературный стиль программирования.

Исходные Julia-скрипты содержат текстовые пояснения в виде комментариев. Это позволяет одновременно использовать их как исполняемые программы и как основу для формирования документации.

Для генерации производных представлений использовался скрипт:

```
scripts/tangle.jl
```

В результате для моделей были получены Julia-скрипты, Jupyter Notebook и документы Quarto.

11 Анализ результатов

11.1 Модель SIR

В базовом эксперименте при

$$\beta = 0.05, \quad c = 10, \quad \gamma = 0.25$$

получено:

$$R_0 = 2.$$

Пиковое число инфицированных составляет около 154.8 человека, а момент достижения пика — примерно 19.1 дня.

Итоговое число выздоровевших составляет около 775.7 человека.

Параметрическое исследование показало существенное влияние коэффициента β на динамику заболевания.

При увеличении β максимальное число инфицированных увеличивается, а пик эпидемии достигается раньше.

Таким образом, увеличение интенсивности передачи инфекции приводит к более быстрому и масштабному распространению заболевания.

11.2 Модель Лотки–Вольтерры

В базовом эксперименте при

$$\alpha = 0.1, \quad \beta = 0.02, \quad \delta = 0.01, \quad \gamma = 0.3$$

наблюдаются циклические изменения численности жертв и хищников.

Стационарная точка для заданных параметров равна:

$$(x^*, y^*) = (30, 5).$$

Полученные графики показывают взаимосвязь между изменением численности двух популяций.

В параметрическом исследовании рассматривались значения:

$$\alpha \in \{0.05, 0.1, 0.2\}.$$

Изменение α влияет на скорость роста популяции жертв и вследствие взаимодействия между видами отражается на динамике хищников.

12 Выводы

В ходе выполнения лабораторной работы были изучены и реализованы две математические модели: SIR и Лотки–Вольтерры.

Для модели SIR было выполнено базовое моделирование и параметрическое исследование коэффициента β для значений:

$$\beta \in \{0.03, 0.05, 0.07\}.$$

Было установлено, что увеличение β приводит к увеличению максимального числа инфицированных и более раннему достижению пика эпидемии.

Для модели Лотки–Вольтерры была исследована динамика взаимодействия популяций жертв и хищников. Были построены графики динамики популяций, фазового портрета, производных, относительных изменений и спектрального анализа.

Также было выполнено параметрическое исследование влияния α для значений:

$$\alpha \in \{0.05, 0.1, 0.2\}.$$

В рамках лабораторной работы были освоены численное решение систем дифференциальных уравнений в Julia, литературное программирование, генерация Jupyter Notebook и документов Quarto, а также проведение параметрических исследований.

Список литературы